# Shor's Algorithm

Number theory, order finding, phase estimation, implementation and cryptography

**ANSWER BOOK**

This book accompanies the separate *Question Paper* (80 questions, 3 hours, 240 marks). It contains

1. the **answer key**, section by section, with the topic tested by every question (page 2),
2. a **topic-wise map** grouping the questions by theme and a **quick-marking grid** (page 3), and
3. the **Answer Bank**: every question restated with the correct option marked ✓ and a complete solution with derivations (from page 4).

## Marking scheme (every question)

Each question has **four** options (A), (B), (C) and (D). **ONLY ONE** of these four options is correct.

| | |
|---|---|
| *Full marks*: | +3 if ONLY the correct option is chosen; |
| *Zero marks*: | 0 if none of the options is chosen (question unanswered); |
| *Negative marks*: | −1 in all other cases (wrong option, or more than one option chosen). |

## How to evaluate

For each section count $N_c$ = number of correct answers and $N_w$ = number of questions answered wrongly (or with more than one bubble darkened). Section marks = $3N_c - N_w$; unanswered questions score 0. The total is out of 240. Every numerical answer was re-computed independently (exact integer and `Fraction` arithmetic, NumPy phase-estimation formulas, checks against the code and recorded outputs of `Shor.ipynb`, and CUDA-Q state-vector and density-matrix simulations with the notebook's circuit builder) by `verify_answers.py` in the source folder.

## Conventions (identical to the Question Paper)

- $a \bmod N$ is the remainder in $\{0, \ldots, N-1\}$; the **order** $r = \mathrm{ord}_N(a)$ is the least $r > 0$ with $a^r \equiv 1 \pmod N$ (defined for $\gcd(a, N) = 1$); $\varphi$ is Euler's totient, $\lambda$ the Carmichael function. "Factors" means non-trivial factors ($1 < p < N$).
- **Reduction:** if $r$ is even and $a^{r/2} \not\equiv -1 \pmod N$, then $\gcd(a^{r/2} - 1, N)$ and $\gcd(a^{r/2} + 1, N)$ are non-trivial factors; otherwise the attempt fails.
- **Circuit:** $t$ counting qubits (Hadamards), work register in $|1\rangle$, controlled $U_{a^{2^j} \bmod N}$ with $U_c\,|y\rangle = |cy \bmod N\rangle$ for $y < N$, inverse QFT, measurement of the counting register giving $y \in \{0, \ldots, 2^t - 1\}$ (most significant bit first). **Post-processing:** `Fraction(y, 2**t).limit_denominator(N)` gives $s/r'$; the order is $r'$ or its smallest multiple $kr'$ with $a^{kr'} \equiv 1$. $y = 0$ gives no candidate.
- **Notebook** (`Shor.ipynb`, defaults): $N = 15$, $a = 7$, $n =$ `N.bit_length()` work qubits, $t = 2n$ counting qubits, multipliers as exact permutation matrices (`cudaq.register_operation`), 1000 shots. **Previously recorded run**: CUDA-Q 0.16.0, CPU target `qpp-cpu`, no GPU; 12 qubits, 51 gates (plus 8 measurements), outcomes $0, 64, 128, 192$; 0.766 of the shots gave $15 = 3 \times 5$. No physical quantum processor was used.
- State-vector memory: 16 bytes per amplitude ($2^q$ amplitudes for $q$ qubits). 1 MB = $10^6$ bytes, 1 GB = $10^9$ bytes.

# Answer key

Answer distribution: (A) 20, (B) 20, (C) 20, (D) 20.

## Section A — Mathematical and Number Theory Foundations

| Q. | Ans. | Topic tested | Q. | Ans. | Topic tested |
|---|---|---|---|---|---|
| 1 | **C** | Euclidean algorithm | 11 | **B** | Failure: $a^{r/2} \equiv -1$ |
| 2 | **D** | Modular inverse | 12 | **B** | Fermat's little theorem |
| 3 | **C** | Euler's totient $\varphi(pq)$ | 13 | **C** | Trial-division bound |
| 4 | **C** | Multiplicative order | 14 | **A** | Success bound $1 - 2^{-(k-1)}$ |
| 5 | **D** | Repeated squaring | 15 | **A** | $\varphi$, order, $\lambda$, inverse |
| 6 | **D** | Square-and-multiply cost | 16 | **B** | Classical factoring complexity |
| 7 | **B** | Orders divide $\lambda(N)$ | 17 | **A** | Non-trivial square root $\Rightarrow$ factor |
| 8 | **B** | Lucky gcd probability | 18 | **C** | Euler's theorem vs order |
| 9 | **D** | Chinese remainder theorem | 19 | **A** | Order of 3 mod 91 |
| 10 | **D** | Square roots of 1 mod 15 | 20 | **A** | Factors of 91 by gcd |

## Section B — Quantum Computing Foundations

| Q. | Ans. | Topic tested | Q. | Ans. | Topic tested |
|---|---|---|---|---|---|
| 21 | **B** | Uniform superposition | 31 | **A** | CNOT cost of a Fredkin gate |
| 22 | **B** | Measuring the work register | 32 | **D** | Implicit measurement |
| 23 | **D** | Unitarity of $U_a$ | 33 | **A** | Notebook qubit count |
| 24 | **D** | Eigenvalues of $U_a$ | 34 | **A** | State-vector memory |
| 25 | **C** | $\|1\rangle$ as sum of eigenvectors | 35 | **C** | Roles of circuit blocks |
| 26 | **C** | Phase kickback | 36 | **D** | Phase gates $R_1(\lambda)$ |
| 27 | **B** | Phases of $U^{2^j}$ | 37 | **C** | Kickback and controls |
| 28 | **C** | Hadamard transform signs | 38 | **D** | Parallelism vs interference |
| 29 | **B** | Two-path interference | 39 | **A** | QPE: exact phase |
| 30 | **A** | Reversible modular exponentiation | 40 | **B** | QPE: inexact phase |

## Section C — Shor's Algorithm Fundamentals

| Q. | Ans. | Topic tested | Q. | Ans. | Topic tested |
|---|---|---|---|---|---|
| 41 | **A** | Steps of Shor's algorithm | 51 | **B** | Exact success, notebook default |
| 42 | **D** | $N = 15$, $a = 2$ | 52 | **A** | Probability $\varphi(r)/r$ |
| 43 | **C** | Failure for $a = N - 1$ | 53 | **A** | Role of the QFT in Shor |
| 44 | **C** | Odd order: retry | 54 | **B** | Skipped identity multipliers |
| 45 | **D** | Lucky base | 55 | **C** | Functions of `Shor.ipynb` |
| 46 | **D** | Register sizes for $N = 33$ | 56 | **C** | Outcomes to fractions |
| 47 | **B** | $N = 15$, $a = 11$ outcomes | 57 | **D** | Prime powers |
| 48 | **B** | Continued fractions, $N = 33$ | 58 | **B** | Uninformative outcomes |
| 49 | **D** | Multiples test | 59 | **C** | $N = 39$: order and factors |
| 50 | **A** | Condition $2^t \geq N^2$ | 60 | **A** | $N = 39$: continued fraction |

## Section D — Advanced Shor's Algorithm and Implementation

| Q. | Ans. | Topic tested | Q. | Ans. | Topic tested |
|---|---|---|---|---|---|
| 61 | **D** | Beauregard qubit count | 71 | **B** | Memory for $N = 1023$ |
| 62 | **C** | Published RSA-2048 estimates | 72 | **A** | Toy RSA |
| 63 | **D** | $O(n^3)$ scaling | 73 | **B** | Post-quantum standards |
| 64 | **C** | Semiclassical QFT | 74 | **A** | Impact on RSA, ECC, AES |
| 65 | **D** | Approximate QFT | 75 | **B** | Fault-tolerance terms |
| 66 | **C** | Controlled phases in the QFT | 76 | **A** | Qubit counts of constructions |
| 67 | **D** | Recorded gate counts | 77 | **B** | Harvest now, decrypt later |
| 68 | **D** | No hardware execution | 78 | **A** | Qubits and repeated squaring |
| 69 | **C** | CNOTs of a controlled multiplier | 79 | **A** | Error-free probability vs success |
| 70 | **B** | Random baseline | 80 | **C** | Interpreting noisy results |

## Topic-wise map of the questions

Every question appears under at least one theme; several appear under two.

| Theme | Question numbers |
|---|---|
| Modular arithmetic, gcd, inverses and totients | 1, 2, 3, 9, 12, 15, 72 |
| Multiplicative order and repeated squaring | 4, 5, 6, 7, 18, 19, 27, 78 |
| Reduction of factoring to order finding | 8, 10, 11, 14, 17, 20, 41, 42, 43, 44, 45, 57, 59 |
| Classical factoring and complexity | 13, 16, 63 |
| Superposition, entanglement and measurement | 21, 22, 30, 32, 38 |
| Unitary $U_a$, eigenvectors and phase kickback | 23, 24, 25, 26, 37 |
| Gates and interference | 28, 29, 31, 36, 66, 69 |
| Phase estimation and the QFT | 35, 39, 40, 53, 64, 65, 66 |
| Outcomes, continued fractions and post-processing | 47, 48, 49, 50, 51, 52, 56, 58, 60 |
| Register sizes, memory and simulation limits | 33, 34, 46, 50, 71, 78 |
| `Shor.ipynb`: code and recorded results | 51, 54, 55, 67, 68 |
| Gate-level circuits and noise | 31, 69, 70, 79, 80 |
| Resource estimates and fault tolerance | 61, 62, 64, 75, 76 |
| RSA, ECC and post-quantum cryptography | 72, 73, 74, 77 |

## Quick-marking grid

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 1 **C** | 2 **D** | 3 **C** | 4 **C** | 5 **D** | 6 **D** | 7 **B** | 8 **B** | 9 **D** | 10 **D** |
| 11 **B** | 12 **B** | 13 **C** | 14 **A** | 15 **A** | 16 **B** | 17 **A** | 18 **C** | 19 **A** | 20 **A** |
| 21 **B** | 22 **B** | 23 **D** | 24 **D** | 25 **C** | 26 **C** | 27 **B** | 28 **C** | 29 **B** | 30 **A** |
| 31 **A** | 32 **D** | 33 **A** | 34 **A** | 35 **C** | 36 **D** | 37 **C** | 38 **D** | 39 **A** | 40 **B** |
| 41 **A** | 42 **D** | 43 **C** | 44 **C** | 45 **D** | 46 **D** | 47 **B** | 48 **B** | 49 **D** | 50 **A** |
| 51 **B** | 52 **A** | 53 **A** | 54 **B** | 55 **C** | 56 **C** | 57 **D** | 58 **B** | 59 **C** | 60 **A** |
| 61 **D** | 62 **C** | 63 **D** | 64 **C** | 65 **D** | 66 **C** | 67 **D** | 68 **D** | 69 **C** | 70 **B** |
| 71 **B** | 72 **A** | 73 **B** | 74 **A** | 75 **B** | 76 **A** | 77 **B** | 78 **A** | 79 **A** | 80 **C** |

# Answer Bank — questions with detailed solutions

## SECTION A · Mathematical and Number Theory Foundations Q.1–20

**Q.1** Single-correct MCQ · Euclidean algorithm

Using the Euclidean algorithm, $\gcd(1071, 462)$ equals

**(A)** 42 **(B)** 7 **(C)** ✓ 21 **(D)** 147

**Answer: (C) Detailed solution**

**Correct answer.** (C) 21

**Concepts required.** The Euclidean algorithm replaces $(a, b)$ by $(b, a \bmod b)$ until the remainder is 0; the last non-zero remainder is the gcd.

**Relevant formulae.** $\gcd(a, b) = \gcd(b, a \bmod b)$: $1071 = 2 \cdot 462 + 147$, $462 = 3 \cdot 147 + 21$, $147 = 7 \cdot 21 + 0$.

**Step-by-step explanation.** $1071 = 2 \cdot 462 + 147$; $462 = 3 \cdot 147 + 21$; $147 = 7 \cdot 21 + 0$. The last non-zero remainder is 21. *Check*: $1071 = 3^2 \cdot 7 \cdot 17$ and $462 = 2 \cdot 3 \cdot 7 \cdot 11$, common part $3 \cdot 7 = 21$. The algorithm needs $O(\log N)$ division steps, which is why the gcd steps of Shor's algorithm are classically cheap.

**Why the other options are incorrect.**

**(A)** 42 does not divide 1071 (odd), so it cannot be a common divisor at all.

**(B)** 7 divides both numbers, but so does $21 = 3 \cdot 7$; 7 is a common divisor, not the greatest one.

**(D)** 147 is the first remainder of the algorithm, not its last non-zero remainder; $147 \nmid 462$.

**Key takeaway.** The final gcd step of Shor's algorithm is this fast classical algorithm: $O(\log N)$ division steps.

**Q.2** Single-correct MCQ · Modular inverse

The multiplicative inverse of 3 modulo 20 (the $b$ with $3b \equiv 1 \pmod{20}$, $0 < b < 20$) is

**(A)** 3 **(B)** 13 **(C)** 17 **(D)** ✓ 7

**Answer: (D) Detailed solution**

**Correct answer.** (D) 7

**Concepts required.** An inverse modulo $m$ exists iff $\gcd(b, m) = 1$; it is found by the extended Euclidean algorithm or by testing.

**Relevant formulae.** $3b \equiv 1 \pmod{20}$: $3 \cdot 7 = 21 = 20 + 1$.

**Step-by-step explanation.** Extended Euclid: $20 = 6 \cdot 3 + 2$, $3 = 1 \cdot 2 + 1$, so $1 = 3 - 2 = 3 - (20 - 6 \cdot 3) = 7 \cdot 3 - 20$. Hence $3 \cdot 7 = 21 \equiv 1 \pmod{20}$, $b = 7$. The inverse exists because $\gcd(3, 20) = 1$; the same condition $\gcd(a, N) = 1$ makes multiplication by $a$ modulo $N$ reversible.

**Why the other options are incorrect.**

**(A)** 3 is the number itself; $3 \cdot 3 = 9 \not\equiv 1 \pmod{20}$.

**(B)** $3 \cdot 13 = 39 \equiv 19 \equiv -1 \pmod{20}$: 13 is the inverse of $-3$, not of 3.

**(C)** $3 \cdot 17 = 51 \equiv 11 \pmod{20}$, which is not 1.

**Key takeaway.** Invertibility of $a$ modulo $N$ is exactly what makes the multiplier $U_a$ a permutation (unitary).

**Q.3** Single-correct MCQ · Euler's totient $\varphi(pq)$

Euler's totient $\varphi(91)$ equals

**(A)** 84 **(B)** 78 **(C)** ✓ 72 **(D)** 90

**Answer: (C) Detailed solution**

**Correct answer.** (C) 72
**Concepts required.** Euler's totient counts the residues coprime to $N$; it is multiplicative for coprime factors.
**Relevant formulae.** $\varphi(pq) = (p-1)(q-1)$ for distinct primes: $\varphi(7 \cdot 13) = 6 \cdot 12 = 72$.
**Step-by-step explanation.** $91 = 7{\cdot}13$ with distinct primes, so $\varphi(91) = (7-1)(13-1) = 6{\cdot}12 = 72$: the number of $1 \le a \le 91$ coprime to 91. $90 = N - 1$ would hold only for a prime $N$.
**Why the other options are incorrect.**
**(A)** $84 = 91 - 7$ removes only the multiples of 13 (7 of them) and forgets the multiples of 7.
**(B)** $78 = 91 - 13$ removes only the multiples of 7 and forgets the multiples of 13.
**(D)** $90 = 91 - 1$ would be $\varphi$ of a prime; $91 = 7 \cdot 13$ is composite.
**Key takeaway.** Knowing $\varphi(N)$ is equivalent to knowing the factors of an RSA modulus.

**Q.4** Single-correct MCQ · Multiplicative order

The multiplicative order of 2 modulo 11 is

**(A)** 11 **(B)** 2 **(C)** ✓ 10 **(D)** 5

**Answer: (C) Detailed solution**

**Correct answer.** (C) 10
**Concepts required.** The order is the least $r > 0$ with $a^r \equiv 1$; it divides $\varphi(N)$ (Lagrange).
**Relevant formulae.** Powers of 2 mod 11: $2, 4, 8, 5, 10, 9, 7, 3, 6, 1$ — the first 1 is at $r = 10$.
**Step-by-step explanation.** Powers of 2 mod 11: $2, 4, 8, 5, 10, 9, 7, 3, 6, 1$ — the first 1 appears at exponent 10. The order divides $\varphi(11) = 10$, so the only candidates are $1, 2, 5, 10$: $2^2 = 4$, $2^5 = 32 \equiv 10 \equiv -1$, so the order is 10 (2 is a primitive root mod 11).
**Why the other options are incorrect.**
**(A)** 11 is the modulus; orders modulo a prime $p$ divide $p - 1 = 10$, so 11 is impossible.
**(B)** $2^2 = 4 \neq 1$; 2 is the base, not its order.
**(D)** $2^5 = 32 \equiv 10 \equiv -1 \pmod{11}$, not 1; 5 divides 10 but is not the order.
**Key takeaway.** Order finding is the hard problem; for tiny $N$ we can list powers, for RSA sizes we cannot.

**Q.5** Single-correct MCQ · Repeated squaring

Using repeated squaring, $3^{13} \bmod 17$ equals

**(A)** 5 **(B)** 3 **(C)** 16 **(D)** ✓ 12

**Answer: (D) Detailed solution**

**Correct answer.** (D) 12
**Concepts required.** Repeated squaring computes $a^{2^k}$ by squaring and multiplies the powers selected by the binary digits of the exponent.
**Relevant formulae.** $13 = 8+4+1$; $3^2 = 9$, $3^4 = 81 \equiv 13$, $3^8 \equiv 169 \equiv 16$; $3^{13} \equiv 16{\cdot}13{\cdot}3 = 624 \equiv 12 \pmod{17}$.
**Step-by-step explanation.** $3^1 = 3$, $3^2 = 9$, $3^4 = 81 \equiv 13$, $3^8 = 169 \equiv 16 \pmod{17}$. With

$13 = 8 + 4 + 1$: $3^{13} \equiv 16 \cdot 13 \cdot 3 = 624 = 36 \cdot 17 + 12 \equiv 12$. *Check*: $3^{16} \equiv 1$ (Fermat), so $3^{13} \equiv 3^{-3} = 27^{-1} \equiv 10^{-1} \equiv 12$ since $10 \cdot 12 = 120 = 7 \cdot 17 + 1$.

**Why the other options are incorrect.**

**(A)** $5 \equiv 3^5 \pmod{17}$ ($243 = 14 \cdot 17 + 5$); it uses the wrong exponent.

**(B)** $3 = 3^1$ is the starting value; it would require $3^{12} \equiv 1$, but the order of 3 modulo 17 is 16.

**(C)** $16 \equiv 3^8$ is only one of the three factors needed for $13 = 8 + 4 + 1$.

**Key takeaway.** Repeated squaring is the classical model for the controlled-$U^{2^j}$ structure of the quantum circuit.

**Q.6** SINGLE-CORRECT MCQ · Square-and-multiply cost

Square-and-multiply computes $a^{1000} \bmod N$ from the binary expansion $1000 = 1111101000_2$. The number of modular multiplications (squarings plus extra multiplications) is

**(A)** 10 **(B)** 999 **(C)** 16 **(D)** ✓ 14

**Answer: (D) Detailed solution**

**Correct answer.** (D) 14

**Concepts required.** Square-and-multiply performs one squaring per bit after the leading one and one extra multiplication per further 1-bit.

**Relevant formulae.** $1000 = 1111101000_2$: 10 bits ⇒ 9 squarings; six 1-bits ⇒ 5 multiplications; total 14.

**Step-by-step explanation.** Left-to-right: start with $a$ (leading bit), then for each of the remaining 9 bits square once (9 squarings) and multiply by $a$ when the bit is 1. The bits after the leading one are 111101000: five 1s, so 5 extra multiplications. Total $9 + 5 = 14$. Naively 999 multiplications would be needed. This is the classical idea behind computing $a^{2^j} \bmod N$ in Shor's circuit.

**Why the other options are incorrect.**

**(A)** 10 is the number of bits; it ignores the extra multiplications for the 1-bits (and overcounts the squarings by one).

**(B)** 999 multiplications is the naive method $a \cdot a \cdots a$, which is exponential in the bit length.

**(C)** 16 counts a squaring and a multiplication for each of the six 1-bits plus extras; only $9 + 5$ operations are needed.

**Key takeaway.** Modular exponentiation is polynomial: $O(n)$ multiplications of $n$-bit numbers, $O(n^3)$ bit operations.

**Q.7** SINGLE-CORRECT MCQ · Orders divide $\lambda(N)$

For $N = 35$, which of the following can *not* be the multiplicative order of any $a$ with $\gcd(a, 35) = 1$?

**(A)** 6 **(B)** ✓ 5 **(C)** 4 **(D)** 12

**Answer: (B) Detailed solution**

**Correct answer.** (B) 5

**Concepts required.** Every order divides the Carmichael function $\lambda(N)$, the exponent of the group $\mathbb{Z}_N^*$.

**Relevant formulae.** $\lambda(35) = \operatorname{lcm}(\lambda(5), \lambda(7)) = \operatorname{lcm}(4, 6) = 12$; possible orders are the divisors of 12: $1, 2, 3, 4, 6, 12$.

**Step-by-step explanation.** By the Chinese remainder theorem the order modulo 35 is $\operatorname{lcm}(r_5, r_7)$ with $r_5 \mid 4$ and $r_7 \mid 6$, so every order divides the Carmichael value $\lambda(35) = \operatorname{lcm}(4, 6) = 12$ (and $\varphi(35) = 24$). Orders 12 (e.g. $a = 2$), 4 (e.g. $a = 8$: $8^2 = 64 \equiv 29$, $8^4 \equiv 1$) and 6 (e.g. $a = 4$:

$4^3 = 64 \equiv 29$, $4^6 \equiv 1$) occur; 5 divides neither 12 nor 24.

**Why the other options are incorrect.**

**(A)** 6 divides 12 and is attained, e.g. by $a = 4$ ($4^3 = 64 \equiv 29 \not\equiv 1$, $4^6 \equiv 1$, $4^2 = 16$).

**(C)** 4 divides 12 and is attained, e.g. by $a = 8$ ($8^2 = 64 \equiv 29$, $8^4 \equiv 29^2 = 841 \equiv 1$).

**(D)** $12 = \lambda(35)$ is attained, e.g. by $a = 3$ ($3^{12} \equiv 1$ and no smaller power is 1).

**Key takeaway.** Orders divide $\lambda(N)$, which divides $\varphi(N)$; 5 does not divide 12.

**Q.8** SINGLE-CORRECT MCQ · Lucky gcd probability

For $N = 15$, a base $a$ is drawn uniformly from $\{2, 3, \ldots, 13\}$. The probability that $\gcd(a, 15) > 1$, so that a factor is found classically without any quantum step, is

**(A)** 7/15 **(B)** ✓ 1/2 **(C)** 8/15 **(D)** 1/3

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** 1/2

**Concepts required.** Before any quantum step, Shor's algorithm computes $\gcd(a, N)$; if it exceeds 1 a factor is found classically.

**Relevant formulae.** In $\{2, \ldots, 13\}$ (12 values) the bases sharing a factor with 15 are $3, 5, 6, 9, 10, 12$: $6/12 = 1/2$.

**Step-by-step explanation.** The multiples of 3 or 5 in $\{2, \ldots, 13\}$ are $3, 5, 6, 9, 10, 12$: six of the twelve values, probability $\frac{1}{2}$. (For RSA-sized $N = pq$ this probability is about $\frac{1}{p} + \frac{1}{q}$, astronomically small — the lucky gcd is only an artefact of tiny examples.)

**Why the other options are incorrect.**

**(A)** 7/15 divides by all 15 residues and counts 0; the bases are drawn from 12 values, 6 of which are lucky.

**(C)** $8/15 = \varphi(15)/15$ is the fraction of coprime residues in $\{0, \ldots, 14\}$, the complement of what is asked, on the wrong set.

**(D)** 1/3 counts only the multiples of 3 $(3, 6, 9, 12)$ or of 5 separately; both kinds must be included.

**Key takeaway.** For tiny $N$ lucky bases are common; for RSA moduli the probability is negligible ($\approx 2/\sqrt{N}$).

**Q.9** SINGLE-CORRECT MCQ · Chinese remainder theorem

The unique $x \in \{0, \ldots, 14\}$ with $x \equiv 2 \pmod 3$ and $x \equiv 3 \pmod 5$ is

**(A)** 13 **(B)** 2 **(C)** 11 **(D)** ✓ 8

**Answer: (D) Detailed solution**

**Correct answer.** **(D)** 8

**Concepts required.** The Chinese remainder theorem: for coprime moduli, the system has a unique solution modulo their product.

**Relevant formulae.** $x \equiv 3 \pmod 5$: $x \in \{3, 8, 13\}$; of these only $8 \equiv 2 \pmod 3$.

**Step-by-step explanation.** Candidates with $x \equiv 3 \pmod 5$: $3, 8, 13$; their residues mod 3 are $0, 2, 1$. So $x = 8$. The Chinese remainder theorem guarantees one solution modulo 15; it is the reason why orders modulo $N = pq$ are least common multiples of the orders modulo $p$ and $q$.

**Why the other options are incorrect.**

**(A)** $13 \equiv 3 \pmod 5$ but $13 \equiv 1 \pmod 3$, not 2.

**(B)** $2 \equiv 2 \pmod 3$ but $2 \not\equiv 3 \pmod 5$.

**(C)** $11 \equiv 2 \pmod 3$ but $11 \equiv 1 \pmod 5$.

**Key takeaway.** CRT explains the four square roots of 1 modulo $pq$: $\pm 1$ modulo each prime independently.

**Q.10** Single-correct MCQ · Square roots of 1 mod 15

The complete set of solutions of $x^2 \equiv 1 \pmod{15}$ with $0 \leq x < 15$ is

**(A)** $\{1, 4, 14\}$ **(B)** $\{1, 14\}$ **(C)** $\{1, 11\}$ **(D)** ✓ $\{1, 4, 11, 14\}$

**Answer: (D) Detailed solution**

**Correct answer.** **(D)** $\{1, 4, 11, 14\}$
**Concepts required.** Modulo $N = pq$, $x^2 \equiv 1$ iff $x \equiv \pm 1$ modulo both $p$ and $q$ (CRT), giving four solutions.
**Relevant formulae.** $1^2 = 1$, $4^2 = 16 \equiv 1$, $11^2 = 121 \equiv 1$, $14^2 = 196 \equiv 1 \pmod{15}$.
**Step-by-step explanation.** By CRT, $x^2 \equiv 1 \bmod 15$ iff $x \equiv \pm 1 \pmod 3$ and $x \equiv \pm 1 \pmod 5$: four combinations, $x = 1, 4, 11, 14$ ($4^2 = 16$, $11^2 = 121 = 8 \cdot 15 + 1$). The non-trivial roots 4 and 11 give factors: $\gcd(4 - 1, 15) = 3$, $\gcd(4 + 1, 15) = 5$. Shor's algorithm finds such a root as $x = a^{r/2}$.
**Why the other options are incorrect.**

**(A)** $\{1, 4, 14\}$ misses $11 = 15 - 4$; roots come in pairs $\pm x$.

**(B)** $\{1, 14\}$ are only the trivial roots $\pm 1$; the non-trivial ones 4 and 11 are what Shor's algorithm exploits.

**(C)** $\{1, 11\}$ misses 4 and 14; $14 \equiv -1$ is always a root.

**Key takeaway.** A non-trivial square root of 1 immediately factors $N$ via $\gcd(x \pm 1, N)$.

**Q.11** Single-correct MCQ · Failure: $a^{r/2} \equiv -1$

For $N = 21$ and $a = 5$, the order $r$ and the outcome of Shor's classical reduction are

**(A)** $r = 12$; it gives the factors 3 and 7

**(B)** ✓ $r = 6$; the reduction fails because $5^3 \equiv -1 \pmod{21}$

**(C)** $r = 3$; it fails because $r$ is odd

**(D)** $r = 6$; it gives the factors 3 and 7

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** $r = 6$; the reduction fails because $5^3 \equiv -1 \pmod{21}$
**Concepts required.** The reduction needs $r$ even and $a^{r/2} \not\equiv -1$; otherwise the attempt fails and a new base is chosen.
**Relevant formulae.** Powers of 5 mod 21: $5, 4, 20, 16, 17, 1 \Rightarrow r = 6$; $5^3 \equiv 20 \equiv -1$.
**Step-by-step explanation.** $5^1 = 5$, $5^2 = 25 \equiv 4$, $5^3 \equiv 20 \equiv -1$, $5^6 \equiv 1$: $r = 6$ (even). But $a^{r/2} = 5^3 \equiv -1$, so $\gcd(5^3 + 1, 21) = \gcd(21, 21) = 21$ and $\gcd(5^3 - 1, 21) = \gcd(19, 21) = 1$: only trivial factors. The algorithm must restart with another $a$.
**Why the other options are incorrect.**

**(A)** 12 is a multiple of the order, not the order; $5^6 \equiv 1$ already.

**(C)** $5^3 \equiv 20 \neq 1$, so 3 is not the order; the order is 6, which is even.

**(D)** With $5^3 \equiv -1$ the gcds are $\gcd(19, 21) = 1$ and $\gcd(21, 21) = 21$: trivial, so no factors are obtained.

**Key takeaway.** Correct order finding can still give a failed attempt; report it as a failure and retry.

**Q.12** Single-correct MCQ · Fermat's little theorem

$2^{100} \bmod 13$ equals

**(A)** 12 **(B)** ✓ 3 **(C)** 1 **(D)** 9

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** 3
**Concepts required.** Fermat's little theorem: $a^{p-1} \equiv 1 \pmod p$ for prime $p \nmid a$; reduce the exponent modulo the order.
**Relevant formulae.** $2^{12} \equiv 1 \pmod{13}$, $100 = 8 \cdot 12 + 4$, so $2^{100} \equiv 2^4 = 16 \equiv 3$.
**Step-by-step explanation.** Fermat: $2^{12} \equiv 1 \pmod{13}$. $100 = 8 \cdot 12 + 4$, so $2^{100} \equiv 2^4 = 16 \equiv 3$. Reducing the exponent modulo the order (or $\varphi$) is what makes $a^x \bmod N$ periodic in $x$.
**Why the other options are incorrect.**
**(A)** $12 \equiv -1$ is $2^6$; the exponent 100 reduces to 4, not 6.
**(C)** 1 would require $12 \mid 100$; $100 = 8 \cdot 12 + 4$.
**(D)** $9 \equiv 2^8 \pmod{13}$ ($256 = 19 \cdot 13 + 9$): it reduces the exponent incorrectly; $100 \bmod 12 = 4$, not 8.
**Key takeaway.** Exponents can be reduced modulo the order — which is exactly why the order is so useful.

**Q.13** Single-correct MCQ · Trial-division bound

Trial division factors $N = 10403$ by testing divisors $d \geq 2$ in increasing order. The largest $d$ that may have to be tested is

**(A)** 103 **(B)** 5201 **(C)** ✓ 101 **(D)** 102

**Answer: (C) Detailed solution**

**Correct answer.** **(C)** 101
**Concepts required.** A composite $N$ has a prime factor $\leq \sqrt{N}$, so trial division stops at $\lfloor \sqrt{N} \rfloor$.
**Relevant formulae.** $\sqrt{10403} = 101.99$; $10403 = 101 \cdot 103$, so $d = 101$ is reached and divides.
**Step-by-step explanation.** If $N$ is composite it has a factor $\leq \sqrt{N}$. $101^2 = 10201 \leq 10403 < 102^2 = 10404$, so testing up to $\lfloor \sqrt{N} \rfloor = 101$ suffices — and $10403 = 101 \times 103$ is found at $d = 101$. For an $n$-bit $N$ this is about $2^{n/2}$ divisions: exponential in $n$.
**Why the other options are incorrect.**
**(A)** 103 is the larger factor; it is found as $N/101$, never by testing.
**(B)** $5201 \approx N/2$ is a far too pessimistic bound; divisors above $\sqrt{N}$ pair with ones below it.
**(D)** $102 > \sqrt{10403}$ is never needed (and is even); the search stops at 101.
**Key takeaway.** Trial division costs $O(\sqrt{N}) = O(2^{n/2})$ — exponential in the bit length.

**Q.14** Single-correct MCQ · Success bound $1 - 2^{-(k-1)}$

For an odd $N$ with $k$ distinct prime factors (not a prime power) and $a$ uniformly random among the residues coprime to $N$, the probability that $r$ is even and $a^{r/2} \not\equiv -1 \pmod{N}$ is at least $1 - 2^{-(k-1)}$. For $N = 105 = 3 \cdot 5 \cdot 7$ this lower bound is

**(A)** ✓ 3/4 **(B)** 1/8 **(C)** 1/2 **(D)** 7/8

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** 3/4
**Concepts required.** For $N$ with $k$ distinct odd prime factors, a random coprime base works with probability at least $1 - 2^{-(k-1)}$.
**Relevant formulae.** $k = 3$: $1 - 2^{-2} = 3/4$.
**Step-by-step explanation.** $k = 3$: $1 - 2^{-2} = \frac{3}{4}$. For RSA moduli ($k = 2$) the bound is $\frac{1}{2}$, so on average at most two bases are needed. This bound comes from the CRT: the reduction fails only if the 2-adic parts of the orders modulo the prime powers all agree.
**Why the other options are incorrect.**
**(B)** $1/8 = 2^{-3}$ confuses the failure bound with the success bound and uses the wrong exponent.
**(C)** 1/2 is the bound for $k = 2$ (an RSA modulus), not for three prime factors.
**(D)** $7/8 = 1 - 2^{-3}$ uses the exponent $k$ instead of $k - 1$.
**Key takeaway.** For an RSA modulus at least half of the bases work, so a few repetitions suffice.

**Q.15** Matching lists · $\varphi$, order, $\lambda$, inverse

Match each quantity for $N = 15$ (List-I) with its value (List-II).

| List-I | | List-II | |
|---|---|---|---|
| **(P)** | $\varphi(15)$ | **(1)** | 13 |
| **(Q)** | order of 4 modulo 15 | **(2)** | 8 |
| **(R)** | Carmichael value $\lambda(15) = \mathrm{lcm}(\lambda(3), \lambda(5))$ | **(3)** | 2 |
| **(S)** | $7^{-1} \bmod 15$ | **(4)** | 4 |

**(A)** ✓ P→2, Q→3, R→4, S→1 **(B)** P→2, Q→3, R→1, S→4
**(C)** P→2, Q→4, R→3, S→1 **(D)** P→4, Q→3, R→2, S→1

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** P→2, Q→3, R→4, S→1
**Concepts required.** $\varphi$, $\lambda$, orders and inverses are basic invariants of $\mathbb{Z}_N^*$; for $N = 15$ they can be computed by hand.
**Relevant formulae.** $\varphi(15) = 2 \cdot 4 = 8$; $4^2 = 16 \equiv 1$ so $\mathrm{ord}(4) = 2$; $\lambda(15) = \mathrm{lcm}(2, 4) = 4$; $7 \cdot 13 = 91 \equiv 1$.
**Step-by-step explanation.**

- $\varphi(15) = 2 \cdot 4 = 8$ (2).
- $4^2 = 16 \equiv 1$: order 2 (3).
- $\lambda(15) = \mathrm{lcm}(2, 4) = 4$ (4): every order divides it.
- $7 \cdot 13 = 91 = 6 \cdot 15 + 1$: $7^{-1} = 13$ (1).

Hence P→2, Q→3, R→4, S→1.
**Why the other options are incorrect.**

**(B)** This gives $\lambda(15) = 13$ and $7^{-1} = 4$; but $7 \cdot 4 = 28 \equiv 13 \not\equiv 1$.

**(C)** This swaps the order of 4 and $\lambda(15)$: $4^2 \equiv 1$, so the order is 2, while $\lambda(15) = 4$.

**(D)** This assigns 4 to $\varphi(15)$; $\varphi(15) = 8$ counts $1, 2, 4, 7, 8, 11, 13, 14$.

**Key takeaway.** Orders divide $\lambda(N)$, which divides $\varphi(N)$: here $2 \mid 4 \mid 8$.

**Q.16** MATCHING LISTS · Classical factoring complexity

Match each classical algorithm (List-I) with its running time for an $n$-bit input $N$ (List-II).

| List-I | | List-II | |
|---|---|---|---|
| **(P)** | trial division | **(1)** | $\exp\left((64/9)^{1/3}(\ln N)^{1/3}(\ln \ln N)^{2/3}(1+o(1))\right)$ (sub-exponential) |
| **(Q)** | Pollard's rho | **(2)** | $O(N^{1/4})$ expected, i.e. $O(2^{n/4})$ (heuristic) |
| **(R)** | general number field sieve | **(3)** | $O(n)$ division steps (polynomial) |
| **(S)** | Euclidean $\gcd(a, N)$ | **(4)** | $O(\sqrt{N}) = O(2^{n/2})$ divisions |

**(A)** P→4, Q→2, R→3, S→1 **(B)** ✓ P→4, Q→2, R→1, S→3

**(C)** P→2, Q→4, R→1, S→3 **(D)** P→4, Q→1, R→2, S→3

**Answer: (B) Detailed solution**

**Correct answer. (B)** P→4, Q→2, R→1, S→3

**Concepts required.** Classical factoring methods range from exponential (trial division, rho) to sub-exponential (GNFS); gcd is polynomial.

**Relevant formulae.** Trial division $O(2^{n/2})$; Pollard rho $O(2^{n/4})$; GNFS $L_N[1/3, (64/9)^{1/3}]$; Euclid $O(n)$ steps.

**Step-by-step explanation.**

- trial division tests divisors up to $\sqrt{N}$ (4);
- Pollard's rho finds a factor $p$ in about $\sqrt{p} \le N^{1/4}$ steps (2);
- the GNFS is the fastest known general method, sub-exponential but super-polynomial (1);
- the gcd is polynomial (3) — not a factoring algorithm on its own.

Hence P→4, Q→2, R→1, S→3. Not all classical factoring is exponential.

**Why the other options are incorrect.**

**(A)** This gives the gcd the GNFS time; the Euclidean algorithm is polynomial, $O(n)$ division steps.

**(C)** This swaps trial division and Pollard's rho; rho finds a factor $p$ in about $\sqrt{p} \le N^{1/4}$ steps, faster than trial division.

**(D)** This gives Pollard's rho the GNFS running time and vice versa; GNFS is the fastest known general method.

**Key takeaway.** The best classical method is sub-exponential; Shor's algorithm is polynomial — on a fault-tolerant machine.

**Q.17** Assertion–Reason · Non-trivial square root ⇒ factor

**Assertion (A):** If $x^2 \equiv 1 \pmod{N}$ but $x \not\equiv \pm 1 \pmod{N}$, then $\gcd(x-1, N)$ is a non-trivial factor of $N$.

**Reason (R):** In that case $N$ divides $(x-1)(x+1)$ but divides neither $x-1$ nor $x+1$.

**(A)** ✓ Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A

**(B)** Both A and R are true, but R is *not* the correct explanation of A

**(C)** A is true, but R is false

**(D)** A is false, but R is true

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A

**Concepts required.** If $N \mid (x-1)(x+1)$ but $N$ divides neither factor, each prime factor of $N$ is split between them.

**Relevant formulae.** $x^2 \equiv 1 \Rightarrow N \mid (x-1)(x+1)$; $x \not\equiv \pm 1 \Rightarrow 1 < \gcd(x \pm 1, N) < N$.

**Step-by-step explanation.** **R is true**: $x^2 - 1 = (x-1)(x+1) \equiv 0$, while $x \not\equiv \pm 1$ means $N \nmid x \mp 1$. **A follows from R**: if $\gcd(x-1, N)$ were 1, then $N \mid (x-1)(x+1)$ would force $N \mid x+1$; if it were $N$, then $N \mid x-1$ — both excluded. So it is a proper divisor. This is exactly Shor's factor-extraction step with $x = a^{r/2}$. Answer: (A).

**Why the other options are incorrect.**

**(B)** is wrong because R is exactly the argument that proves A: a divisor of a product that divides neither factor must share a proper factor with each.

**(C)** is wrong because R is true: $x^2 - 1 = (x-1)(x+1) \equiv 0$, and $x \not\equiv \pm 1$ means $N$ divides neither factor.

**(D)** is wrong because A is true: e.g. $x = 4$, $N = 15$ gives $\gcd(3, 15) = 3$.

**Key takeaway.** This is the heart of the reduction: $x = a^{r/2}$ is a square root of 1.

**Q.18** Statement I / II · Euler's theorem vs order

**Statement I:** If $\gcd(a, N) = 1$, then $a^{\varphi(N)} \equiv 1 \pmod{N}$.

**Statement II:** The multiplicative order of every $a$ coprime to $N$ equals $\varphi(N)$.

**(A)** Both Statement I and Statement II are true

**(B)** Both Statement I and Statement II are false

**(C)** ✓ Statement I is true, but Statement II is false

**(D)** Statement I is false, but Statement II is true

**Answer: (C) Detailed solution**

**Correct answer.** **(C)** Statement I is true, but Statement II is false

**Concepts required.** Euler's theorem gives an exponent that always works; the order is the smallest one and is usually a proper divisor of $\varphi(N)$.

**Relevant formulae.** $a^{\varphi(N)} \equiv 1$; $\mathrm{ord}(a) \mid \lambda(N) \mid \varphi(N)$; e.g. $\mathrm{ord}_{15}(4) = 2 \neq 8$.

**Step-by-step explanation.** **I is true** (Euler's theorem). **II is false**: the order only *divides* $\varphi(N)$; e.g. 4 has order 2 modulo 15 while $\varphi(15) = 8$. Finding the actual order is the hard problem that Shor's quantum period finding solves. Answer: (C).

**Why the other options are incorrect.**

**(A)** is wrong because Statement II is false: $\mathrm{ord}_{15}(4) = 2$ while $\varphi(15) = 8$.

**(B)** is wrong because Statement I is Euler's theorem, which holds for every $a$ coprime to $N$.

**(D)** is wrong because Statement I is true and Statement II false, not the other way round.

**Key takeaway.** If orders were always $\varphi(N)$, knowing them would directly reveal $\varphi(N)$; in reality they only divide it.

**Paragraph for Questions 19 and 20**

Shor's reduction is applied by hand to $N = 91 = 7 \times 13$ with the base $a = 3$ (note $\gcd(3, 91) = 1$). Recall that the order $r$ of $a$ modulo $N$ is the least $r > 0$ with $a^r \equiv 1 \pmod{N}$, and that if $r$ is even and $a^{r/2} \not\equiv -1 \pmod{N}$, then $\gcd(a^{r/2} \pm 1, N)$ are non-trivial factors.

**Q.19** PARAGRAPH BASED · Order of 3 mod 91

The order $r$ of 3 modulo 91 is

**(A)** ✓ 6 **(B)** 3 **(C)** 72 **(D)** 12

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** 6
**Concepts required.** The order is the first exponent where the powers return to 1.
**Relevant formulae.** $3, 9, 27, 81, 243 \equiv 61, 183 \equiv 1 \pmod{91}$ $(729 = 8 \cdot 91 + 1)$: $r = 6$.
**Step-by-step explanation.** By CRT, $r = \operatorname{lcm}(r_7, r_{13})$. Mod 7: $3, 2, 6, 4, 5, 1$: order 6. Mod 13: $3, 9, 27 \equiv 1$: order 3. So $r = \operatorname{lcm}(6, 3) = 6$. *Check*: $3^6 = 729 = 8 \cdot 91 + 1$. ($72 = \varphi(91)$ is only a multiple of the order.)
**Why the other options are incorrect.**
**(B)** $3^3 = 27 \neq 1$; 3 is half of the order.
**(C)** $72 = \varphi(91)$ is an exponent that works, not the least one.
**(D)** 12 is a multiple of the order ($3^{12} \equiv 1$) but not the least one.
**Key takeaway.** Order = smallest exponent; multiples and $\varphi(N)$ also satisfy $a^x \equiv 1$.

**Q.20** PARAGRAPH BASED · Factors of 91 by gcd

The values of $\gcd(3^{r/2} - 1, 91)$ and $\gcd(3^{r/2} + 1, 91)$ are, respectively,

**(A)** ✓ 13 and 7 **(B)** 7 and 13 **(C)** 1 and 91 **(D)** 26 and 28

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** 13 and 7
**Concepts required.** With $r$ even and $a^{r/2} \not\equiv -1$, the gcds with $a^{r/2} \mp 1$ give the factors.
**Relevant formulae.** $3^{r/2} = 27$; $\gcd(26, 91) = 13$, $\gcd(28, 91) = 7$; $7 \cdot 13 = 91$.
**Step-by-step explanation.** $r = 6$ is even and $3^3 = 27 \not\equiv -1 \equiv 90$. $\gcd(26, 91) = 13$ (since $91 = 7 \cdot 13$, $26 = 2 \cdot 13$) and $\gcd(28, 91) = 7$ ($28 = 4 \cdot 7$). Both factors are found: $91 = 7 \times 13$. The order of the two numbers matters: $a^{r/2} - 1 = 26$ shares 13 with 91.
**Why the other options are incorrect.**
**(B)** 7 and 13 are the correct factors in the wrong order: $\gcd(26, 91) = 13$ and $\gcd(28, 91) = 7$.
**(C)** 1 and 91 would be the trivial result for $a^{r/2} \equiv -1$; but $27 \not\equiv 90$.
**(D)** 26 and 28 are $a^{r/2} \mp 1$ themselves; the gcd with 91 is still needed.
**Key takeaway.** The quantum part ends with $r$; the factors come from two classical gcds.

## SECTION B · Quantum Computing Foundations Q.21–40

**Q.21** Single-correct MCQ · Uniform superposition

After $H^{\otimes 8}$ acts on $|0\rangle^{\otimes 8}$ (the counting register of the notebook for $N = 15$), the amplitude of each basis state $|x\rangle$ is

**(A)** $1/\sqrt{8}$ **(B)** ✓ $1/16$ **(C)** $1/8$ **(D)** $1/256$

**Answer: (B) Detailed solution**

**Correct answer.** (B) $1/16$

**Concepts required.** The Hadamard transform of $|0\rangle^{\otimes t}$ is the uniform superposition of all $2^t$ basis states.

**Relevant formulae.** $H^{\otimes t}|0\rangle^{\otimes t} = 2^{-t/2}\sum_x |x\rangle$; $t = 8$: amplitude $1/16$, probability $1/256$.

**Step-by-step explanation.** $H^{\otimes t}|0\rangle^{\otimes t} = \frac{1}{\sqrt{2^t}}\sum_{x=0}^{2^t-1}|x\rangle$. With $t = 8$: $\frac{1}{\sqrt{256}} = \frac{1}{16}$. The probabilities are $\frac{1}{256}$ each. This superposition is the input on which the modular exponentiation acts "for all $x$ at once".

**Why the other options are incorrect.**

**(A)** $1/\sqrt{8}$ is the amplitude for $t = 3$ qubits.

**(C)** $1/8$ would be the amplitude for $t = 6$ ($2^{-3}$); for $t = 8$ it is $2^{-4}$.

**(D)** $1/256$ is the probability $|1/16|^2$, not the amplitude.

**Key takeaway.** Amplitudes are $2^{-t/2}$; probabilities $2^{-t}$.

**Q.22** Single-correct MCQ · Measuring the work register

The state $\frac{1}{\sqrt{8}}\sum_{x=0}^{7}|x\rangle\,|2^x \bmod 15\rangle$ is prepared and the *second* register is measured with result 4. The probability of this result and the post-measurement state of the first register are

**(A)** $1/4$; $\frac{1}{\sqrt{2}}(|2\rangle - |6\rangle)$ **(B)** ✓ $1/4$; $\frac{1}{\sqrt{2}}(|2\rangle + |6\rangle)$

**(C)** $1/2$; $\frac{1}{\sqrt{2}}(|2\rangle + |6\rangle)$ **(D)** $1/8$; $|2\rangle$

**Answer: (B) Detailed solution**

**Correct answer.** (B) $1/4$; $\frac{1}{\sqrt{2}}(|2\rangle + |6\rangle)$

**Concepts required.** Measuring one register of an entangled state keeps only the compatible terms of the other, renormalised.

**Relevant formulae.** $2^x \bmod 15$: $1, 2, 4, 8, 1, 2, 4, 8$; value 4 at $x = 2, 6$: $P = 2/8$, state $(|2\rangle+|6\rangle)/\sqrt{2}$.

**Step-by-step explanation.** $2^x \bmod 15$ for $x = 0, \ldots, 7$: $1, 2, 4, 8, 1, 2, 4, 8$. The value 4 occurs for $x = 2$ and $x = 6$, so $P = 2 \times \frac{1}{8} = \frac{1}{4}$, and the first register collapses to $\frac{1}{\sqrt{2}}(|2\rangle + |6\rangle)$ (equal positive amplitudes, renormalised). It is periodic with period $r = 4$ and an unknown offset — the QFT removes the offset from the probabilities.

**Why the other options are incorrect.**

**(A)** The amplitudes before the measurement are all $+1/\sqrt{8}$; no minus sign can appear.

**(C)** $1/2$ counts two of the four values as matching; only $x = 2, 6$ of 8 terms give 4.

**(D)** $1/8$ and $|2\rangle$ forget that the function is periodic: $x = 6$ gives $2^6 = 64 \equiv 4$ as well.

**Key takeaway.** The collapsed first register is periodic with period $r$ and a random offset.

**Q.23** Single-correct MCQ · Unitarity of $U_a$

Why does the operator $U_a|y\rangle = |ay \bmod N\rangle$ (for $0 \le y < N$; identity for $y \ge N$) require $\gcd(a, N) = 1$ to be a valid quantum gate?

**(A)** because otherwise $U_a$ would not be Hermitian

**(B)** because the eigenvalues of $U_a$ must be real

**(C)** because a quantum gate must map $|0\rangle$ to $|0\rangle$

**(D)** ✓ only then is $y \mapsto ay \bmod N$ a permutation of $\{0, \dots, N-1\}$, so $U_a$ is a permutation matrix and hence unitary

**Answer: (D) Detailed solution**

**Correct answer.** **(D)** only then is $y \mapsto ay \bmod N$ a permutation of $\{0, \dots, N-1\}$, so $U_a$ is a permutation matrix and hence unitary

**Concepts required.** A permutation matrix is unitary; a non-injective map of basis states is not.

**Relevant formulae.** $\gcd(a, N) = 1 \iff a$ invertible mod $N \iff y \mapsto ay \bmod N$ bijective on $\{0, \dots, N-1\}$.

**Step-by-step explanation.** If $\gcd(a, N) = d > 1$, two inputs collide: e.g. $a = 3$, $N = 15$: $3 \cdot 0 \equiv 3 \cdot 5 \equiv 0$. A non-injective map cannot be unitary. For $\gcd(a, N) = 1$, $a$ is invertible modulo $N$, the map is a bijection, and a permutation matrix satisfies $U^\dagger U = I$. Gates need not be Hermitian, and their eigenvalues are complex phases.

**Why the other options are incorrect.**

**(A)** Quantum gates must be unitary, not Hermitian; $U_a$ is generally not Hermitian (it cycles states).

**(B)** Unitary eigenvalues are phases $e^{i\phi}$; $U_a$ has complex eigenvalues $e^{2\pi i s/r}$.

**(C)** Gates need not fix $|0\rangle$; $U_a$ happens to fix $|0\rangle$, but that is not the requirement.

**Key takeaway.** Coprimality makes modular multiplication reversible, hence implementable as a quantum gate.

**Q.24** Single-correct MCQ · Eigenvalues of $U_a$

On the subspace spanned by $\{|1\rangle, |2\rangle, |4\rangle, |8\rangle\}$, the operator $U_2|y\rangle = |2y \bmod 15\rangle$ has the eigenvalues

**(A)** $1, 2, 4, 8$

**(B)** $e^{2\pi i/15}, e^{4\pi i/15}, e^{8\pi i/15}, e^{16\pi i/15}$

**(C)** $1, -1$ (each twice)

**(D)** ✓ $1, i, -1, -i$

**Answer: (D) Detailed solution**

**Correct answer.** **(D)** $1, i, -1, -i$

**Concepts required.** A cyclic permutation of length $r$ has the $r$-th roots of unity as eigenvalues.

**Relevant formulae.** $U_2$: $1 \to 2 \to 4 \to 8 \to 1$, $r = 4$; eigenvalues $e^{2\pi i s/4} = 1, i, -1, -i$.

**Step-by-step explanation.** $U_2$ cycles $|1\rangle \to |2\rangle \to |4\rangle \to |8\rangle \to |1\rangle$: a cyclic shift of length $r = 4$, so $U_2^4 = I$ there and the eigenvalues are the fourth roots of unity $e^{2\pi i s/4}$, $s = 0, 1, 2, 3$. In general the eigenvalues on the orbit of $|1\rangle$ are $e^{2\pi i s/r}$: the order $r$ is encoded in the eigenphases $s/r$ — this is what phase estimation reads out.

**Why the other options are incorrect.**

**(A)** $1, 2, 4, 8$ are the basis labels on the orbit, not eigenvalues; unitary eigenvalues have modulus 1.

**(B)** Phases with denominator 15 use $N$ instead of the order $r = 4$.

**(C)** $\pm 1$ would require $U_2^2 = I$, but $U_2^2|1\rangle = |4\rangle \neq |1\rangle$.

**Key takeaway.** The order is hidden in the eigenphases $s/r$ of $U_a$.

**Q.25** Single-correct MCQ · $|1\rangle$ as sum of eigenvectors

With $|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} |a^k \bmod N\rangle$, the superposition $\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |u_s\rangle$ equals

**(A)** $|0\rangle$ **(B)** $|a\rangle$

**(C)** ✓ $|1\rangle$ **(D)** $\frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} |y\rangle$

**Answer: (C) Detailed solution**

**Correct answer.** **(C)** $|1\rangle$

**Concepts required.** The eigenvectors $|u_s\rangle$ are Fourier transforms of the orbit; summing over $s$ inverts the transform.

**Relevant formulae.** $\sum_{s=0}^{r-1} e^{-2\pi i s k/r} = r\,\delta_{k,0}$, so $\frac{1}{\sqrt{r}} \sum_s |u_s\rangle = |a^0\rangle = |1\rangle$.

**Step-by-step explanation.** $\frac{1}{\sqrt{r}} \sum_s |u_s\rangle = \frac{1}{r} \sum_k \left( \sum_s e^{-2\pi i s k/r} \right) |a^k\rangle = \sum_k \delta_{k,0} |a^k\rangle = |a^0\rangle = |1\rangle$, because the sum of $r$-th roots of unity vanishes unless $k \equiv 0$. That is why the work register is initialised to $|1\rangle$: it is an equal superposition of all eigenvectors, and QPE returns a random $s/r$.

**Why the other options are incorrect.**

**(A)** $|0\rangle$ is outside the orbit $\{a^k \bmod N\}$ of $|1\rangle$, so it cannot be a combination of the $|u_s\rangle$.

**(B)** $|a\rangle = |a^1\rangle$ would need the phase factors $e^{2\pi i s/r}$ inserted before summing.

**(D)** The uniform superposition over all residues is not in the span of one orbit (it includes non-coprime values).

**Key takeaway.** Initialising the work register to $|1\rangle$ prepares all eigenvectors at once, so QPE yields a random $s/r$.

**Q.26** Single-correct MCQ · Phase kickback

A control qubit in $|+\rangle$ controls a unitary $U$ acting on its eigenvector $|u\rangle$, $U|u\rangle = e^{i\phi}|u\rangle$, with $\phi = 2\pi/3$. Afterwards, the expectation value $\langle X \rangle$ of the control qubit is

**(A)** $1/2$ **(B)** $1$ **(C)** ✓ $-1/2$ **(D)** $-\sqrt{3}/2$

**Answer: (C) Detailed solution**

**Correct answer.** **(C)** $-1/2$

**Concepts required.** Phase kickback moves the eigenphase of the target onto the $|1\rangle$ branch of the control.

**Relevant formulae.** $|+\rangle|u\rangle \mapsto \frac{1}{\sqrt{2}}(|0\rangle + e^{i\phi}|1\rangle)|u\rangle$; $\langle X \rangle = \cos\phi$, $\langle Y \rangle = \sin\phi$.

**Step-by-step explanation.** Phase kickback: $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|u\rangle \mapsto \frac{1}{\sqrt{2}}(|0\rangle + e^{i\phi}|1\rangle)|u\rangle$ — the work register is unchanged, the phase moves to the control. For $\frac{1}{\sqrt{2}}(|0\rangle + e^{i\phi}|1\rangle)$, $\langle X \rangle = \cos\phi = \cos\frac{2\pi}{3} = -\frac{1}{2}$ (and $\langle Y \rangle = \sin\phi = \frac{\sqrt{3}}{2}$).

**Why the other options are incorrect.**

**(A)** $1/2 = \cos(\pi/3)$ uses half of the phase.

**(B)** $\langle X \rangle = 1$ is the value before the controlled gate; the kickback rotates the state.

**(D)** $-\sqrt{3}/2$ mixes up sine and cosine; $\sin(2\pi/3) = +\sqrt{3}/2$ is $\langle Y \rangle$.

**Key takeaway.** The control qubit records the phase; the eigenvector target is untouched.

**Q.27** Single-correct MCQ · Phases of $U^{2^j}$

In phase estimation, counting qubit $j$ controls $U^{2^j}$. If $U|u\rangle = e^{2\pi i/6}|u\rangle$, the phase kicked back onto counting qubit $j = 2$ is

**(A)** $2\pi/3$ **(B)** ✓ $4\pi/3$ **(C)** $\pi/3$ **(D)** $2\pi$

**Answer: (B) Detailed solution**

**Correct answer.** (B) $4\pi/3$
**Concepts required.** Applying $U^{2^j}$ to an eigenvector multiplies its phase by $2^j$.
**Relevant formulae.** Phase on qubit $j$: $2\pi\varphi \cdot 2^j$; $\varphi = 1/6$, $j = 2$: $2\pi \cdot 4/6 = 4\pi/3$.
**Step-by-step explanation.** $U^{2^j}|u\rangle = e^{2\pi i \cdot 2^j/6}|u\rangle$; for $j = 2$: $2\pi \cdot \frac{4}{6} = \frac{4\pi}{3}$ (equivalently $-\frac{2\pi}{3}$). Each counting qubit receives the phase doubled relative to the previous one — the binary-fraction pattern that the inverse QFT decodes.
**Why the other options are incorrect.**
**(A)** $2\pi/3$ is the phase for $j = 1$ ($U^2$).
**(C)** $\pi/3 = 2\pi/6$ is the phase for $j = 0$ ($U$ itself).
**(D)** $2\pi$ would correspond to $U^6$; $2^j = 4$, not 6.
**Key takeaway.** Counting qubit $j$ carries the binary digit pattern $2^j\varphi$ mod 1.

**Q.28** Single-correct MCQ · Hadamard transform signs

$(H \otimes H)|11\rangle$ equals

**(A)** $\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$ **(B)** $\frac{1}{2}(|00\rangle - |01\rangle - |10\rangle - |11\rangle)$
**(C)** ✓ $\frac{1}{2}(|00\rangle - |01\rangle - |10\rangle + |11\rangle)$ **(D)** $\frac{1}{\sqrt{2}}(|00\rangle - |11\rangle)$

**Answer: (C) Detailed solution**

**Correct answer.** (C) $\frac{1}{2}(|00\rangle - |01\rangle - |10\rangle + |11\rangle)$
**Concepts required.** $H^{\otimes n}|x\rangle = 2^{-n/2}\sum_y (-1)^{x\cdot y}|y\rangle$.
**Relevant formulae.** $x = 11$: signs $(-1)^{y_1+y_2}$: $+, -, -, +$ for $00, 01, 10, 11$.
**Step-by-step explanation.** $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$, so $H|1\rangle \otimes H|1\rangle = \frac{1}{2}(|0\rangle - |1\rangle)(|0\rangle - |1\rangle) = \frac{1}{2}(|00\rangle - |01\rangle - |10\rangle + |11\rangle)$. In general $H^{\otimes n}|x\rangle = 2^{-n/2}\sum_y (-1)^{x\cdot y}|y\rangle$: signs depend on the parity of $x \cdot y$.
**Why the other options are incorrect.**
**(A)** All plus signs is $(H \otimes H)|00\rangle$.
**(B)** The last sign is $(-1)^{1+1} = +1$, not $-1$.
**(D)** A Bell-type state cannot arise from local Hadamards on a product state.
**Key takeaway.** Hadamard transforms encode parities in signs — the simplest Fourier transform.

**Q.29** Single-correct MCQ · Two-path interference

Two computational paths lead to the same outcome with amplitudes $\frac{1}{2}$ and $\frac{1}{2}e^{i\theta}$. For $\theta = 2\pi/3$ the probability of that outcome is

**(A)** 0 **(B)** ✓ 1/4 **(C)** 3/4 **(D)** 1/2

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** 1/4

**Concepts required.** Probabilities come from the squared modulus of the *sum* of amplitudes of indistinguishable paths.

**Relevant formulae.** $\left|\frac{1}{2}(1+e^{i\theta})\right|^2 = \cos^2(\theta/2)$; $\theta = 2\pi/3$: $\cos^2(\pi/3) = 1/4$.

**Step-by-step explanation.** Amplitudes add before squaring: $\left|\frac{1}{2}(1+e^{i\theta})\right|^2 = \frac{1}{4}|1+e^{i\theta}|^2 = \frac{1}{4}(2+2\cos\theta) = \cos^2\frac{\theta}{2} = \cos^2\frac{\pi}{3} = \frac{1}{4}$. Interference concentrates probability where the phases agree; in Shor's algorithm the inverse QFT arranges this for $y \approx k2^t/r$.

**Why the other options are incorrect.**

**(A)** 0 needs $\theta = \pi$ (complete destructive interference).

**(C)** $3/4 = \sin^2(\pi/3)$ is the probability of the other output port.

**(D)** 1/2 adds probabilities ($\frac{1}{4} + \frac{1}{4}$) instead of amplitudes, ignoring interference.

**Key takeaway.** Interference, not parallelism alone, makes the right answer likely.

**Q.30** Single-correct MCQ · Reversible modular exponentiation

Why does Shor's circuit compute $|x\rangle|1\rangle \mapsto |x\rangle|a^x \bmod N\rangle$ instead of $|x\rangle \mapsto |a^x \bmod N\rangle$ in place?

**(A)** ✓ the map $x \mapsto a^x \bmod N$ is not injective ($x$ and $x+r$ give the same value), so it cannot be a unitary on a single register; keeping $x$ makes it reversible

**(B)** because measurement requires two registers

**(C)** because the Hadamard gates must act on a different register than the QFT

**(D)** because a single register cannot hold numbers larger than $N$

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** the map $x \mapsto a^x \bmod N$ is not injective ($x$ and $x+r$ give the same value), so it cannot be a unitary on a single register; keeping $x$ makes it reversible

**Concepts required.** Unitary operations are invertible; a many-to-one function must keep its input to be reversible.

**Relevant formulae.** $a^{x+r} \equiv a^x$; $|x\rangle|y\rangle \mapsto |x\rangle|y\,a^x \bmod N\rangle$ is reversible.

**Step-by-step explanation.** Quantum gates are unitary, hence invertible. Since $a^{x+r} \equiv a^x$, an in-place map would send different inputs to the same output. Keeping the input $x$ and writing the result into a second register (here by multiplying the work register $|1\rangle$ by $a^x$) gives a reversible map and entangles the registers: $\sum_x |x\rangle|a^x \bmod N\rangle$.

**Why the other options are incorrect.**

**(B)** Measurement can be performed on any register; it does not require a second one.

**(C)** The Hadamards and the QFT both act on the counting register; this is not why two registers are used.

**(D)** Register size is not the issue: $a^x \bmod N < N$ always fits in $n$ qubits.

**Key takeaway.** Reversibility forces the input to be kept, which entangles the two registers.

**Q.31** Single-correct MCQ · CNOT cost of a Fredkin gate

A controlled-SWAP (Fredkin) gate is compiled as $\mathrm{CNOT}_{b\to a} \cdot \mathrm{Toffoli}_{c,a\to b} \cdot \mathrm{CNOT}_{b\to a}$, and each Toffoli uses the standard decomposition with 6 CNOTs. The number of CNOTs per controlled-SWAP is

**(A)** ✓ 8 **(B)** 6 **(C)** 3 **(D)** 9

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** 8

**Concepts required.** A Fredkin gate can be built from two CNOTs around a Toffoli; a Toffoli needs 6 CNOTs in the standard decomposition.

**Relevant formulae.** $\mathrm{CSWAP}_{c;a,b} = \mathrm{CX}_{b\to a}\,\mathrm{CCX}_{c,a\to b}\,\mathrm{CX}_{b\to a}$: $1+6+1=8$.

**Step-by-step explanation.** $1+6+1=8$ CNOTs (plus single-qubit $H$, $T$, $T^\dagger$ gates). A plain SWAP needs 3 CNOTs, but the control turns the middle CNOT into a Toffoli. Gate-level modular multipliers are built from many such gates, which is why compiled Shor circuits are much larger than the notebook's matrix gates.

**Why the other options are incorrect.**

**(B)** 6 counts only the Toffoli and forgets the two outer CNOTs.

**(C)** 3 CNOTs make an uncontrolled SWAP; the control requires the Toffoli.

**(D)** $9 = 3 \times 3$ treats every CNOT of a SWAP as a 3-CNOT gate; only the middle CNOT must become a (6-CNOT) Toffoli.

**Key takeaway.** Controlled arithmetic is costly: every controlled SWAP is a Toffoli plus two CNOTs.

**Q.32** Single-correct MCQ · Implicit measurement

`Shor.ipynb` measures only the counting register. Compared with also measuring the work register first, the distribution of the counting outcome $y$ is

**(A)** different: without measuring the work register no periodic structure appears

**(B)** different: the peaks become twice as wide

**(C)** undefined, because entangled registers cannot be measured separately

**(D)** ✓ exactly the same: a measurement of the other register that is never used cannot change the marginal statistics of $y$

**Answer: (D) Detailed solution**

**Correct answer.** **(D)** exactly the same: a measurement of the other register that is never used cannot change the marginal statistics of $y$

**Concepts required.** Measuring a subsystem that is never used again does not change the reduced state of the rest.

**Relevant formulae.** $\rho_{\text{count}} = \mathrm{Tr}_{\text{work}}\,|\Psi\rangle\langle\Psi| = \sum_w P(w)\rho_{\text{count}|w}$.

**Step-by-step explanation.** The work register is not acted on after the controlled multiplications. Measuring it (or not) only changes which conditional state the counting register is in; averaged over the unseen result, the reduced state of the counting register — and hence $P(y)$ — is identical (principle of implicit measurement). Textbook derivations "measure the second register" only to simplify the algebra.

**Why the other options are incorrect.**

**(A)** The periodic structure is in the entangled state already; the measurement of the work register only simplifies textbook algebra.

**(B)** Peak widths depend on $t$ and $r$, not on whether the idle work register is measured.

**(C)** Entangled registers can be measured separately; the marginal statistics are well defined.

**Key takeaway.** Principle of implicit (deferred) measurement: unused registers may be measured or ignored.

**Q.33** Single-correct MCQ · Notebook qubit count

In `Shor.ipynb` the work register has $n =$ `N.bit_length()` qubits and the counting register $t = 2n$. The total number of qubits for $N = 143$ is

**(A)** ✓ 24 **(B)** 21 **(C)** 16 **(D)** 22

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** 24
**Concepts required.** The notebook uses $n = \lceil \log_2(N+1) \rceil$ work qubits and $t = 2n$ counting qubits: $3n$ in total.
**Relevant formulae.** $143 = 10001111_2$: $n = 8$, $3n = 24$.
**Step-by-step explanation.** $143 = 10001111_2$ has 8 bits, so $n = 8$, $t = 16$ and $t + n = 24$ (the notebook's comment lists "143 -> 24"). 16 forgets the work register; $22 = 2\lceil \log_2 143 \rceil + 6$ has no basis.
**Why the other options are incorrect.**
**(B)** 21 would be $3 \cdot 7$, but $143 > 127 = 2^7 - 1$ needs 8 bits.
**(C)** 16 is only the counting register $t = 2n$.
**(D)** 22 does not follow from $t = 2n$ with $n = 8$; no formula of the notebook gives it.
**Key takeaway.** Qubits scale as $3n$ here: three per bit of $N$.

**Q.34** Single-correct MCQ · State-vector memory

The memory of a double-precision state vector (16 bytes per amplitude) for the 24-qubit circuit of the previous question is about

**(A)** ✓ 268 MB **(B)** 4.3 GB **(C)** 384 bytes **(D)** 16.8 MB

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** 268 MB
**Concepts required.** A state vector of $q$ qubits has $2^q$ complex amplitudes of 16 bytes (complex128).
**Relevant formulae.** $M = 16 \cdot 2^q$ bytes; $q = 24$: 268 435 456 B $\approx$ 268 MB.
**Step-by-step explanation.** $16 \times 2^{24} = 268\,435\,456$ bytes $\approx$ 268 MB (the notebook's table). $384 = 16 \times 24$ is linear; 16.8 MB is $2^{20} \times 16$ (20 qubits); 4.3 GB corresponds to 28 qubits. Each extra bit of $N$ adds 3 qubits, i.e. multiplies the memory by 8.
**Why the other options are incorrect.**
**(B)** 4.3 GB is $16 \cdot 2^{28}$, the memory of 28 qubits.
**(C)** 384 bytes grows linearly with the number of qubits; state vectors grow exponentially.
**(D)** 16.8 MB is $16 \cdot 2^{20}$, the memory of 20 qubits.
**Key takeaway.** Each extra qubit doubles the memory; each extra bit of $N$ multiplies it by 8.

**Q.35** MATCHING LISTS · Roles of circuit blocks

Match each operation of the order-finding circuit (List-I) with its role (List-II).

| List-I | | List-II | |
|---|---|---|---|
| **(P)** | $H$ on every counting qubit | **(1)** | turns the phases $2\pi \cdot 2^j s/r$ into a binary estimate $y \approx 2^t s/r$ |
| **(Q)** | controlled $U_{a^{2^j} \bmod N}$ | **(2)** | prepares the work register in $\|1\rangle$, an equal superposition of the eigenvectors $\|u_s\rangle$ |
| **(R)** | inverse QFT on the counting register | **(3)** | creates the uniform superposition over all exponents $x$ |
| **(S)** | $X$ on the last work qubit | **(4)** | performs modular exponentiation / kicks the eigenphases back onto the counting qubits |

**(A)** P→3, Q→4, R→2, S→1
**(B)** P→4, Q→3, R→1, S→2
**(C)** ✓ P→3, Q→4, R→1, S→2
**(D)** P→3, Q→1, R→4, S→2

**Answer: (C) Detailed solution**

**Correct answer.** **(C)** P→3, Q→4, R→1, S→2
**Concepts required.** Order finding: superposition, controlled modular exponentiation (kickback), inverse QFT, measurement.
**Relevant formulae.** $\frac{1}{\sqrt{2^t}} \sum_x |x\rangle |1\rangle \to \frac{1}{\sqrt{2^t}} \sum_x |x\rangle |a^x\rangle \to$ peaks at $y \approx k2^t/r$.
**Step-by-step explanation.**

- Hadamards: superposition over $x$ (3);
- controlled powers of $U_a$: $|x\rangle |1\rangle \mapsto |x\rangle |a^x\rangle$, equivalently phase kickback (4);
- inverse QFT: interference that converts the periodic phase pattern into peaks at $y \approx k2^t/r$ (1);
- $X$ on the least significant work qubit: $|0001\rangle = |1\rangle$ (2).

Hence P→3, Q→4, R→1, S→2.
**Why the other options are incorrect.**
**(A)** This swaps the roles of the inverse QFT and the $X$ gate; $X$ only prepares $|1\rangle$ in the work register.
**(B)** This gives the Hadamards the role of modular exponentiation and vice versa; the Hadamards only create the superposition.
**(D)** This gives the controlled multiplications the decoding role of the inverse QFT and the QFT the exponentiation.
**Key takeaway.** Know what each block does: it is the basis for every resource and error discussion.

**Q.36** Matching lists · Phase gates $R_1(\lambda)$

The phase gate is $R_1(\lambda) = \mathrm{diag}(1, e^{i\lambda})$ (controlled versions appear in the QFT). Match each angle (List-I) with the named gate (List-II).

| List-I | | List-II | |
|---|---|---|---|
| **(P)** | $R_1(\pi)$ | **(1)** | $S^\dagger$ |
| **(Q)** | $R_1(\pi/2)$ | **(2)** | $T$ |
| **(R)** | $R_1(\pi/4)$ | **(3)** | $Z$ |
| **(S)** | $R_1(-\pi/2)$ | **(4)** | $S$ |

**(A)** P→3, Q→2, R→4, S→1
**(B)** P→4, Q→3, R→2, S→1
**(C)** P→3, Q→4, R→1, S→2
**(D)** ✓ P→3, Q→4, R→2, S→1

**Answer: (D) Detailed solution**

**Correct answer.** (D) P→3, Q→4, R→2, S→1
**Concepts required.** $R_1(\lambda) = \mathrm{diag}(1, e^{i\lambda})$ contains $Z$, $S$, $T$ and their inverses as special cases.
**Relevant formulae.** $e^{i\pi} = -1$, $e^{i\pi/2} = i$, $e^{i\pi/4}$, $e^{-i\pi/2} = -i$.
**Step-by-step explanation.** $R_1(\pi) = \mathrm{diag}(1,-1) = Z$ (3); $R_1(\pi/2) = \mathrm{diag}(1,i) = S$ (4); $R_1(\pi/4) = \mathrm{diag}(1, e^{i\pi/4}) = T$ (2); $R_1(-\pi/2) = \mathrm{diag}(1,-i) = S^\dagger$ (1). The inverse QFT of the notebook uses `cr1(-pi/2**(k-j))`: controlled $S^\dagger$, $T^\dagger$, ...
**Why the other options are incorrect.**
**(A)** This exchanges $S$ and $T$: $R_1(\pi/2) = \mathrm{diag}(1,i) = S$, while $T$ has $e^{i\pi/4}$.
**(B)** This assigns $R_1(\pi)$ to $S$; $R_1(\pi) = \mathrm{diag}(1,-1) = Z$.
**(C)** This exchanges $T$ and $S^\dagger$; $R_1(-\pi/2)$ has $-i$ on the diagonal, which is $S^\dagger$.
**Key takeaway.** The inverse QFT uses controlled $R_1(-\pi/2^k)$: controlled $S^\dagger$, $T^\dagger$ and finer rotations.

**Q.37** Assertion–Reason · Kickback and controls

**Assertion (A):** When the target register is in an eigenvector of $U$, a controlled-$U$ gate leaves the target register unchanged.

**Reason (R):** Controlled gates never change the state of their control qubit.

**(A)** Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A
**(B)** Both A and R are true, but R is *not* the correct explanation of A
**(C)** ✓ A is true, but R is false
**(D)** A is false, but R is true

**Answer: (C) Detailed solution**

**Correct answer.** (C) A is true, but R is false
**Concepts required.** Phase kickback: a controlled-$U$ on an eigenvector changes the control (in superposition), not the target.
**Relevant formulae.** $(\alpha|0\rangle + \beta|1\rangle)|u\rangle \mapsto (\alpha|0\rangle + \beta e^{i\phi}|1\rangle)|u\rangle$.
**Step-by-step explanation.** **A is true**: $|c\rangle|u\rangle \mapsto |c\rangle\, e^{ic\phi}|u\rangle$ — the target stays $|u\rangle$, only a phase appears. **R is false**: that phase belongs to the control branch $|1\rangle$, so a control in superposition changes, $|+\rangle \mapsto \frac{1}{\sqrt{2}}(|0\rangle + e^{i\phi}|1\rangle)$ (phase kickback). This is precisely how phase estimation and Shor's algorithm extract information. Answer: (C).
**Why the other options are incorrect.**
**(A)** is wrong because R is false: the control's relative phase changes, as phase kickback shows.

**(B)** is wrong for the same reason: R is false, so it cannot be true but irrelevant.

**(D)** is wrong because A is true: the target remains the eigenvector $|u\rangle$.

**Key takeaway.** In QPE the information flows from target to control — the opposite of the classical intuition.

**Q.38** Statement I / II · Parallelism vs interference

**Statement I:** Because the circuit evaluates $a^x \bmod N$ for all $x$ in superposition, measuring the registers directly reveals the period $r$.

**Statement II:** Interference produced by the inverse QFT is needed to turn the periodic structure into high-probability measurement outcomes.

**(A)** Both Statement I and Statement II are true

**(B)** Both Statement I and Statement II are false

**(C)** Statement I is true, but Statement II is false

**(D)** ✓ Statement I is false, but Statement II is true

**Answer: (D) Detailed solution**

**Correct answer.** (D) Statement I is false, but Statement II is true

**Concepts required.** Quantum parallelism produces a superposition, but a measurement returns one sample; interference extracts global properties.

**Relevant formulae.** Measuring $\sum_x |x\rangle\,|f(x)\rangle$ gives one pair $(x, f(x))$; the inverse QFT concentrates amplitude at $y \approx k2^t/r$.

**Step-by-step explanation.** **I is false**: a measurement returns one random $x$ and one value $a^x \bmod N$ — no more information than a single classical evaluation. Quantum parallelism alone does not solve factoring. **II is true**: the inverse QFT makes the amplitudes interfere constructively near $y = k2^t/r$, so that one measurement carries information about $r$. Answer: (D).

**Why the other options are incorrect.**

**(A)** is wrong because Statement I is false: a direct measurement yields one random evaluation, not the period.

**(B)** is wrong because Statement II is true: interference via the inverse QFT is the essential step.

**(C)** has the truth values reversed: I is false and II is true.

**Key takeaway.** Parallelism alone does not solve factoring; interference does the work.

**Paragraph for Questions 39 and 40**

Quantum phase estimation (QPE) with $t = 3$ counting qubits is applied to a unitary $U$ and an eigenvector $|u\rangle$ with $U|u\rangle = e^{2\pi i\varphi}|u\rangle$, $0 \le \varphi < 1$. After the Hadamards, the controlled-$U^{2^j}$ gates and the inverse QFT, the counting register is measured and gives $y \in \{0, \dots, 7\}$ with probability

$$P(y) = \left|\frac{1}{8}\sum_{x=0}^{7} e^{2\pi i x(\varphi - y/8)}\right|^2 = \frac{\sin^2\left(8\pi(\varphi - y/8)\right)}{64\,\sin^2\left(\pi(\varphi - y/8)\right)} \quad (P = 1 \text{ if } \varphi = y/8).$$

The estimate of the phase is $y/8$; $y$ is written in binary with the most significant bit first.

**Q.39** PARAGRAPH BASED · QPE: exact phase

For $\varphi = 3/8$ the measurement gives

**(A)** ✓ $y = 3$ (bits 011) with probability 1
**(B)** $y = 6$ (bits 110) with probability 1
**(C)** $y = 3$ or $y = 4$ with probability 1/2 each
**(D)** $y = 3$ with probability 1/2

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** $y = 3$ (bits 011) with probability 1
**Concepts required.** QPE is exact when the phase has a $t$-bit binary expansion.
**Relevant formulae.** $\varphi = 3/8 = 0.011_2$; $P(y) = 1$ for $y = 8\varphi = 3$.
**Step-by-step explanation.** $\varphi = 3/8 = 0.011_2$ is exactly representable with three bits, so all terms in the sum have phase 0 for $y = 3$: $P(3) = 1$, and $P(y) = 0$ for every other $y$ (the geometric sum of eighth roots of unity vanishes). 110 is the bit-reversed string — the inverse QFT's final SWAPs make the most significant bit come first.
**Why the other options are incorrect.**
**(B)** 110 is the bit-reversed string; the notebook's inverse QFT includes the SWAPs, so $y = 3$ is read MSB first.
**(C)** Splitting between 3 and 4 happens for $\varphi$ halfway between grid points (7/16), not for 3/8.
**(D)** For an exactly representable phase all amplitude interferes constructively at $y = 3$; probability 1, not 1/2.
**Key takeaway.** If $r$ divides $2^t$, Shor's outcomes are exactly $s2^t/r$ (as for $N = 15$).

**Q.40** PARAGRAPH BASED · QPE: inexact phase

For $\varphi = 1/3$ the most likely outcome and its probability are

**(A)** $y = 3$ with $P \approx 0.41$
**(B)** ✓ $y = 3$ with $P \approx 0.69$
**(C)** $y = 3$ with $P = 1$
**(D)** $y = 2$ or 3 with $P = 1/2$ each

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** $y = 3$ with $P \approx 0.69$
**Concepts required.** For phases that are not multiples of $2^{-t}$ the outcome distribution is a peak around $2^t\varphi$.
**Relevant formulae.** $\delta = 1/3 - 3/8 = -1/24$; $P(3) = \sin^2(\pi/3)/(64\sin^2(\pi/24)) \approx 0.688$.
**Step-by-step explanation.** $8\varphi = 2.67$, nearest integer $y = 3$, $\delta = \varphi - 3/8 = -\frac{1}{24}$. $P(3) = \frac{\sin^2(8\pi/24)}{64\sin^2(\pi/24)} = \frac{\sin^2(\pi/3)}{64\sin^2(7.5°)} = \frac{0.75}{64\times 0.017037} = \frac{0.75}{1.0904} \approx 0.688$. Because 1/3 has no finite binary expansion, the probability spreads over neighbouring $y$; $\frac{4}{\pi^2} \approx 0.41$ is only the worst-case lower bound for the nearest outcome.
**Why the other options are incorrect.**
**(A)** $4/\pi^2 \approx 0.405$ is only the worst-case lower bound for the nearest outcome, attained when $\varphi$ lies

halfway.

**(C)** Probability 1 needs an exactly representable phase; 1/3 has the infinite binary expansion $0.0101\ldots_2$.

**(D)** $P(2) \approx 0.17$ and $P(3) \approx 0.69$: $8\varphi = 2.67$ is closer to 3 than to 2, so the two are far from equal.

**Key takeaway.** Peaks have finite width; continued fractions turn the nearest $y$ into $s/r$.

## SECTION C · Shor's Algorithm Fundamentals Q.41–60

**Q.41** Single-correct MCQ · Steps of Shor's algorithm

Which sequence describes Shor's factoring algorithm for an odd composite $N$ that is not a prime power?

**(A)** ✓ pick random $a$; if $\gcd(a, N) > 1$ stop (factor found); find the order $r$ of $a$ by quantum phase estimation and continued fractions; if $r$ is even and $a^{r/2} \not\equiv -1$, output $\gcd(a^{r/2} \pm 1, N)$; otherwise repeat

**(B)** pick random $a$; measure $a^x$ mod $N$ in superposition; the most frequent value is a factor

**(C)** find the order $r$ by quantum phase estimation; the factors are $r$ and $N/r$

**(D)** apply the QFT to $|N\rangle$; measure; the outcome is a factor of $N$

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** pick random $a$; if $\gcd(a, N) > 1$ stop (factor found); find the order $r$ of $a$ by quantum phase estimation and continued fractions; if $r$ is even and $a^{r/2} \not\equiv -1$, output $\gcd(a^{r/2} \pm 1, N)$; otherwise repeat

**Concepts required.** Shor's algorithm is a classical reduction from factoring to order finding, with a quantum subroutine for the order only.

**Relevant formulae.** $r = \mathrm{ord}_N(a)$; if $2 \mid r$ and $a^{r/2} \not\equiv -1$: $p = \gcd(a^{r/2}-1, N)$, $q = \gcd(a^{r/2}+1, N)$.

**Step-by-step explanation.** The quantum computer is used only for *order finding*; everything else — the gcd test, continued fractions, the checks on $r$ and the final $\gcd(a^{r/2} \pm 1, N)$ — is classical. Option 3 confuses the order with a factor (for $N = 15$, $a = 7$: $r = 4$ does not divide 15).

**Why the other options are incorrect.**

**(B)** Measuring $a^x$ mod $N$ gives one random value of the function; no interference step and no factor appear.

**(C)** The order is not a factor: for $N = 15$, $a = 7$, $r = 4 \nmid 15$; the factors come from $\gcd(a^{r/2} \pm 1, N)$.

**(D)** A QFT of the number $N$ itself produces no factor; Shor applies the QFT to the counting register after modular exponentiation.

**Key takeaway.** Quantum part: period finding. Classical part: everything else.

---

**Q.42** Single-correct MCQ · $N = 15$, $a = 2$

For $N = 15$ and $a = 2$, the order $r$ and the factors obtained from $\gcd(a^{r/2} \pm 1, N)$ are

**(A)** $r = 8$; 3 and 5

**(B)** $r = 2$; 3 and 5

**(C)** $r = 4$; 1 and 15

**(D)** ✓ $r = 4$; $\gcd(3, 15) = 3$, $\gcd(5, 15) = 5$

**Answer: (D) Detailed solution**

**Correct answer.** **(D)** $r = 4$; $\gcd(3, 15) = 3$, $\gcd(5, 15) = 5$

**Concepts required.** For a small $N$ the order can be read off the list of powers; the gcds then give the factors.

**Relevant formulae.** $2, 4, 8, 16 \equiv 1 \pmod{15}$: $r = 4$; $2^2 = 4$: $\gcd(3, 15) = 3$, $\gcd(5, 15) = 5$.

**Step-by-step explanation.** $2^1, 2^2, 2^3, 2^4 \equiv 2, 4, 8, 1$, so $r = 4$. $2^{r/2} = 4$, $4 \not\equiv -1 \equiv 14$. $\gcd(4 - 1, 15) = 3$ and $\gcd(4 + 1, 15) = 5$, and $3 \times 5 = 15$. $r = 8$ is a multiple of the order, not the order (the smallest positive exponent).

**Why the other options are incorrect.**

**(A)** 8 is a multiple of the order; the order is the smallest exponent with $a^r \equiv 1$.

**(B)** $2^2 = 4 \neq 1$, so $r \neq 2$; the order is 4 (the factors are right, but for the wrong reason).

**(C)** 1 and 15 are the trivial gcds that occur only for $a^{r/2} \equiv -1$; here $4 \not\equiv 14$.
**Key takeaway.** $N = 15$, $a = 2$ behaves like the notebook default $a = 7$: $r = 4$, factors $3 \times 5$.

**Q.43** Single-correct MCQ · Failure for $a = N - 1$

For $N = 15$ and $a = 14$, the order finding succeeds perfectly. The classical post-processing then

**(A)** succeeds with factors 3 and 5

**(B)** fails because $r$ is odd

**(C)** ✓ fails: $r = 2$ but $14^1 \equiv -1 \pmod{15}$, so $\gcd(14 \pm 1, 15)$ gives only the trivial factors 15 and 1; another $a$ is needed

**(D)** fails because $\gcd(14, 15) = 1$

**Answer: (C) Detailed solution**

**Correct answer.** **(C)** fails: $r = 2$ but $14^1 \equiv -1 \pmod{15}$, so $\gcd(14 \pm 1, 15)$ gives only the trivial factors 15 and 1; another $a$ is needed
**Concepts required.** The reduction fails when $a^{r/2} \equiv -1 \pmod{N}$, even if the order is found correctly.
**Relevant formulae.** $14 \equiv -1$, $14^2 \equiv 1$: $r = 2$, $a^{r/2} = 14 \equiv -1$; $\gcd(13, 15) = 1$, $\gcd(15, 15) = 15$.
**Step-by-step explanation.** $14^2 = 196 = 13 \cdot 15 + 1$, so $r = 2$ (even). But $a^{r/2} = 14 \equiv -1$: $\gcd(13, 15) = 1$ and $\gcd(15, 15) = 15$ — trivial. For any $N$, $a = N - 1$ is always such a failing base. $\gcd(a, N) = 1$ is required, not a reason for failure.
**Why the other options are incorrect.**

**(A)** 3 and 5 cannot appear: with $a^{r/2} \equiv -1$ both gcds are trivial.

**(B)** $r = 2$ is even; the odd-order failure is a different case (e.g. $a = 4$ modulo 21).

**(D)** $\gcd(a, N) = 1$ is the normal case required for order finding, not a cause of failure.

**Key takeaway.** $a = N - 1$ always fails: a failed attempt must be reported, then another base tried.

**Q.44** Single-correct MCQ · Odd order: retry

For $N = 21$ and $a = 4$, the outcome of the classical checks is

**(A)** $r = 6$, factors 3 and 7

**(B)** $r = 3$, factors $\gcd(4^1 - 1, 21) = 3$ and $\gcd(4^1 + 1, 21) = 1$

**(C)** ✓ $r = 3$ is odd, so $a^{r/2}$ is not an integer power; the attempt fails and a new $a$ is chosen

**(D)** $\gcd(4, 21) = 7$, a lucky factor

**Answer: (C) Detailed solution**

**Correct answer.** **(C)** $r = 3$ is odd, so $a^{r/2}$ is not an integer power; the attempt fails and a new $a$ is chosen
**Concepts required.** An odd order gives no square root $a^{r/2}$ in the integers; the attempt fails.
**Relevant formulae.** $4, 16, 64 \equiv 1 \pmod{21}$: $r = 3$.
**Step-by-step explanation.** $4^1, 4^2, 4^3 \equiv 4, 16, 64 = 3 \cdot 21 + 1$, so $r = 3$, odd. The reduction needs $r$ even (then $(a^{r/2} - 1)(a^{r/2} + 1) \equiv 0$). A failed attempt must be reported as a failure, not as a success; one simply repeats with another base ($a = 2$ gives $r = 6$ and the factors 3, 7).
**Why the other options are incorrect.**

**(A)** $4^3 \equiv 1$, so the order is 3, not 6; 6 is the order of $a = 2$ modulo 21.

**(B)** The reduction is not applied for odd $r$; "$4^1$" is not $a^{r/2}$ for $r = 3$, and a gcd of 1 is no factor anyway.

**(D)** $\gcd(4, 21) = 1$; 4 shares no factor with $21 = 3 \cdot 7$.

**Key takeaway.** Odd order ⇒ retry with a new base; never present it as a success.

**Q.45** Single-correct MCQ · Lucky base

For $N = 21$ the randomly chosen base is $a = 6$. What happens?

**(A)** order finding gives $r = 6$ and the factors 3, 7

**(B)** $U_6$ is a valid unitary and phase estimation returns $r = 2$

**(C)** the algorithm fails because $a$ is even

**(D)** ✓ $\gcd(6, 21) = 3$ is already a non-trivial factor; no quantum computation is needed

**Answer: (D) Detailed solution**

**Correct answer.** **(D)** $\gcd(6, 21) = 3$ is already a non-trivial factor; no quantum computation is needed

**Concepts required.** The first step of the algorithm is the classical test $\gcd(a, N)$.

**Relevant formulae.** $\gcd(6, 21) = 3$, $21/3 = 7$.

**Step-by-step explanation.** The first classical step is $\gcd(a, N)$. Here $\gcd(6, 21) = 3$, so $21 = 3 \times 7$ immediately (a "lucky" base). Moreover $U_6$ would not be unitary ($6 \cdot 0 \equiv 6 \cdot 7 \equiv 0 \pmod{21}$), and 6 has no order modulo 21. Even bases are fine in general (e.g. $a = 2$).

**Why the other options are incorrect.**

**(A)** Order finding is not defined for $a$ not coprime to $N$: no power of 6 is $\equiv 1 \pmod{21}$.

**(B)** $U_6$ maps $|0\rangle$ and $|7\rangle$ to $|0\rangle$: it is not unitary, so phase estimation cannot be applied.

**(C)** Even bases are allowed; $a = 2$ works for $N = 21$. The point is the common factor 3.

**Key takeaway.** Always compute $\gcd(a, N)$ first — sometimes it already solves the problem.

**Q.46** Single-correct MCQ · Register sizes for $N = 33$

With the register sizes of `Shor.ipynb` ($n =$ `N.bit_length()`, $t = 2n$), the circuit for $N = 33$ uses

**(A)** 33 qubits

**(B)** 15 qubits ($n = 5$, $t = 10$)

**(C)** 12 qubits

**(D)** ✓ 18 qubits ($n = 6$, $t = 12$)

**Answer: (D) Detailed solution**

**Correct answer.** **(D)** 18 qubits ($n = 6$, $t = 12$)

**Concepts required.** Register sizes: $n =$ bit length of $N$, $t = 2n$ counting qubits, $3n$ in total.

**Relevant formulae.** $33 = 100001_2$: $n = 6$, $t = 12$, total 18.

**Step-by-step explanation.** $33 = 100001_2$ has 6 bits, so $n = 6$, $t = 12$ and $t + n = 18$. Five bits only reach $31 < 33$, so $n = 5$ is too small. The count is $3n$ in this construction — far from the $N$ qubits of option 4.

**Why the other options are incorrect.**

**(A)** The qubit count grows with $\log_2 N$, not with $N$; 33 qubits would be $3 \times 11$ bits.

**(B)** $n = 5$ bits only represent $0, \ldots, 31$, which does not contain all residues modulo 33.

**(C)** 12 is only the counting register $t = 2n$.

**Key takeaway.** $3n$ qubits in the notebook's construction; better constructions need about $2n$.

**Q.47** SINGLE-CORRECT MCQ · $N = 15$, $a = 11$ outcomes

For $N = 15$, $a = 11$ and $t = 8$ counting qubits, the ideal outcome distribution and the result of post-processing $y = 128$ are

**(A)** a uniform distribution, because 11 is prime

**(B)** ✓ peaks at $y \in \{0, 128\}$ with probability $1/2$ each; $128/256 = 1/2 \Rightarrow r = 2$, $\gcd(10, 15) = 5$, $\gcd(12, 15) = 3$

**(C)** peaks at $y \in \{0, 128\}$; $r = 2$, but $11 \equiv -1$, so the factors are trivial

**(D)** peaks at $y \in \{0, 64, 128, 192\}$; $r = 4$

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** peaks at $y \in \{0, 128\}$ with probability $1/2$ each; $128/256 = 1/2 \Rightarrow r = 2$, $\gcd(10, 15) = 5$, $\gcd(12, 15) = 3$

**Concepts required.** For $r \mid 2^t$ the outcomes are exactly $k2^t/r$, each with probability $1/r$.

**Relevant formulae.** $11^2 \equiv 1$: $r = 2$, peaks at 0 and 128; $\gcd(11 \mp 1, 15) = 5, 3$.

**Step-by-step explanation.** $11^2 = 121 = 8 \cdot 15 + 1$, so $r = 2$ and the peaks are at $k \cdot 256/2 = 0, 128$, each with probability $\frac{1}{2}$ (exact, since $r$ divides $2^t$). $y = 128$ gives $s/r = 1/2$, $r = 2$; $11 \not\equiv -1 \equiv 14$, and $\gcd(10, 15) = 5$, $\gcd(12, 15) = 3$. $y = 0$ gives no information.

**Why the other options are incorrect.**

**(A)** Primality of the base is irrelevant; the distribution depends only on the order $r$.

**(C)** $11 \not\equiv -1 \equiv 14 \pmod{15}$; the gcds 5 and 3 are non-trivial.

**(D)** Four peaks would mean $r = 4$; but $11^2 = 121 \equiv 1$, so $r = 2$.

**Key takeaway.** The number of peaks is the order; their spacing is $2^t/r$.

**Q.48** SINGLE-CORRECT MCQ · Continued fractions, $N = 33$

For $N = 33$, $a = 5$ and $t = 12$, the measurement gives $y = 1229$. The continued-fraction step and the factors are

**(A)** $1229/4096 \approx 3/10 \Rightarrow r = 10$; $5^5 \equiv -1$, trivial factors

**(B)** ✓ $1229/4096 \approx 3/10 \Rightarrow r = 10$; $5^5 \equiv 23$, $\gcd(22, 33) = 11$, $\gcd(24, 33) = 3$

**(C)** $1229/4096 \approx 1/3 \Rightarrow r = 3$; the attempt fails

**(D)** $1229/4096 \approx 3/10 \Rightarrow r = 3$; factors 3 and 11

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** $1229/4096 \approx 3/10 \Rightarrow r = 10$; $5^5 \equiv 23$, $\gcd(22, 33) = 11$, $\gcd(24, 33) = 3$

**Concepts required.** Continued fractions find the best approximation $s/r$ with $r \leq N$ to $y/2^t$.

**Relevant formulae.** $1229/4096 = 0.30005 \approx 3/10$; $5^{10} \equiv 1$, $5^5 \equiv 23$; $\gcd(22, 33) = 11$, $\gcd(24, 33) = 3$.

**Step-by-step explanation.** $1229/4096 = 0.30005$; `limit_denominator(33)` gives $3/10$ (within $\frac{1}{2 \cdot 4096}$). $5^{10} \equiv 1 \pmod{33}$ and no smaller power is 1, so $r = 10$. $5^5 = 3125 = 94 \cdot 33 + 23$; $23 \neq 32$; $\gcd(22, 33) = 11$, $\gcd(24, 33) = 3$. The numerator 3 is $s$, not the order.

**Why the other options are incorrect.**

**(A)** $5^5 \equiv 23$, and $-1 \equiv 32$; so the factors are non-trivial.

**(C)** $1/3 = 0.333$ is much farther from 0.30005 than $3/10$; continued fractions give $3/10$.

**(D)** The numerator 3 is the random index $s$; the order is the denominator 10.

**Key takeaway.** Read the order from the denominator, then verify $a^r \equiv 1$.

**Q.49** Single-correct MCQ · Multiples test

For $N = 33$, $a = 5$, $t = 12$, the outcome is $y = 2048$. The procedure of `candidate_orders` returns

**(A)** the order 2, because the denominator is always the order

**(B)** the order 5

**(C)** no candidate, because $y = 2^{t-1}$ is never useful

**(D)** ✓ the fraction 1/2 and, after testing multiples of 2, the order 10 (since $5^2, 5^4, 5^6, 5^8 \not\equiv 1$, $5^{10} \equiv 1$)

**Answer: (D) Detailed solution**

**Correct answer.** **(D)** the fraction 1/2 and, after testing multiples of 2, the order 10 (since $5^2, 5^4, 5^6, 5^8 \not\equiv 1$, $5^{10} \equiv 1$)
**Concepts required.** If $\gcd(s, r) > 1$ the reduced fraction has a denominator that divides $r$; testing multiples recovers $r$.
**Relevant formulae.** $2048/4096 = 1/2 = 5/10$; test $2k$: $5^{10} \equiv 1$ first at $k = 5$.
**Step-by-step explanation.** $2048/4096 = 1/2$: the measured $s/r = 5/10$ has $\gcd(s, r) = 5$, so it reduces to 1/2. Testing $2k$: $5^2 \equiv 25$, $5^4 \equiv 31$, $5^6 \equiv 16$, $5^8 \equiv 4$, $5^{10} \equiv 1$ — the first multiple that works is 10. Without the multiples test this outcome would be lost.
**Why the other options are incorrect.**
**(A)** The denominator is the order only if $\gcd(s, r) = 1$; here $s = 5$, $r = 10$ share the factor 5.
**(B)** $5^5 \equiv 23 \neq 1$, so 5 is not the order; also 5 is not a multiple of the denominator 2.
**(C)** $y = 2^{t-1}$ is useful whenever $r$ is even: it gives 1/2, and the multiples test finds $r$.
**Key takeaway.** The multiples test turns many "partial" outcomes into successes.

**Q.50** Single-correct MCQ · Condition $2^t \geq N^2$

The textbook condition $2^t \geq N^2$ guarantees that continued fractions recover $s/r$ from the nearest outcome. For $N = 33$, the smallest such $t$ and the value used by `Shor.ipynb` are

**(A)** ✓ $t = 11$; the notebook uses $t = 2n = 12$
**(B)** $t = 10$; the notebook uses 12
**(C)** $t = 6$; the notebook uses 12
**(D)** $t = 12$; the notebook uses 11

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** $t = 11$; the notebook uses $t = 2n = 12$
**Concepts required.** With $2^t \geq N^2$, the outcome nearest $s2^t/r$ lies within $\frac{1}{2r^2}$ of $s/r$, so continued fractions find it.
**Relevant formulae.** $N^2 = 1089$; $2^{10} = 1024 < 1089 \leq 2048 = 2^{11}$.
**Step-by-step explanation.** $N^2 = 1089$: $2^{10} = 1024 < 1089 \leq 2^{11} = 2048$, so $t = 11$. The notebook uses $t = 2 \cdot$`bit_length` $= 12$, which always satisfies $2^{2n} \geq N^2$ because $N < 2^n$. Then two distinct fractions with denominators $\leq N$ differ by more than $1/N^2 \geq 1/2^t$.
**Why the other options are incorrect.**
**(B)** $2^{10} = 1024 < 1089 = N^2$, so $t = 10$ is too small.
**(C)** $t = 6$ is the work register size $n$; $2^6 = 64 \ll N^2$.
**(D)** This swaps the two values: the minimum is 11, and the notebook's $t = 2n$ gives 12.
**Key takeaway.** $t = 2n$ always satisfies $2^t \geq N^2$, with up to one qubit to spare.

**Q.51** SINGLE-CORRECT MCQ · Exact success, notebook default

For the notebook's default ($N = 15$, $a = 7$, $t = 8$) the ideal outcomes are $y \in \{0, 64, 128, 192\}$, each with probability $1/4$. Using the multiples test, the exact probability that one shot yields the factors is

**(A)** 1
**(B)** ✓ $3/4$ — only $y = 0$ fails
**(C)** $1/2$ — only $y = 64$ and 192 succeed
**(D)** $1/4$ — only $y = 64$ succeeds

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** $3/4$ — only $y = 0$ fails
**Concepts required.** Count the outcomes whose post-processing yields the factors, weighted by their probabilities.
**Relevant formulae.** $y = 64, 192 \to r = 4$; $y = 128 \to 1/2 \to$ multiples $\to 4$; $y = 0 \to$ nothing: $3/4$.
**Step-by-step explanation.** $y = 64$, 192 give $1/4$, $3/4$, i.e. $r = 4$ directly; $y = 128$ gives $1/2$, and the multiples test finds 4; $y = 0$ gives no denominator. $7^2 = 49 \equiv 4$, $\gcd(3, 15) = 3$, $\gcd(5, 15) = 5$. Hence $3/4$; the recorded run gave 0.766 (finite-shot fluctuation around 0.75).
**Why the other options are incorrect.**
**(A)** Success probability 1 is impossible: the outcome $y = 0$ occurs with probability $1/4$ and carries no information.
**(C)** $1/2$ forgets that $y = 128$ succeeds through the multiples test.
**(D)** $1/4$ counts one successful outcome; three of the four peaks succeed.
**Key takeaway.** Recorded 0.766 vs exact 0.75: the difference is shot noise, not a better algorithm.

**Q.52** SINGLE-CORRECT MCQ · Probability $\varphi(r)/r$

Suppose $r = 12$ divides $2^t$, so every outcome is exactly $y = s2^t/r$ with $s$ uniform in $\{0, \ldots, 11\}$. *Without* the multiples test, the fraction of shots whose reduced denominator equals $r$ is

**(A)** ✓ $\varphi(12)/12 = 1/3$ **(B)** $1/12$ **(C)** $1/2$ **(D)** $11/12$

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** $\varphi(12)/12 = 1/3$
**Concepts required.** The reduced fraction $s/r$ keeps the denominator $r$ iff $\gcd(s, r) = 1$.
**Relevant formulae.** $P = \varphi(r)/r$; $\varphi(12) = |\{1, 5, 7, 11\}| = 4$, $4/12 = 1/3$.
**Step-by-step explanation.** The reduced fraction $s/12$ has denominator 12 exactly when $\gcd(s, 12) = 1$: $s \in \{1, 5, 7, 11\}$, i.e. $\varphi(12) = 4$ of 12 values. In general the probability is $\varphi(r)/r = \Omega(1/\log\log r)$, so $O(\log\log N)$ repetitions (or the multiples/lcm trick) suffice.
**Why the other options are incorrect.**
**(B)** $1/12$ counts only $s = 1$; $s = 5, 7, 11$ are also coprime to 12.
**(C)** $1/2$ is the fraction of odd $s$; $s = 3, 9$ are odd but share the factor 3 with 12.
**(D)** $11/12$ counts every $s \neq 0$; most of them share a factor with 12.
**Key takeaway.** $\varphi(r)/r = \Omega(1/\log\log r)$: a few repetitions suffice.

**Q.53** SINGLE-CORRECT MCQ · Role of the QFT in Shor

What is the role of the (inverse) quantum Fourier transform *within* Shor's algorithm?

**(A)** ✓ it is a subroutine of phase estimation that turns the periodic phase pattern on the counting register into peaks near $y = k2^t/r$; it does not factor $N$ by itself

**(B)** it is the whole of Shor's algorithm; the remaining steps are optional

**(C)** it computes $a^x$ mod $N$

**(D)** it finds the factors $p$ and $q$ directly from $|N\rangle$

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** it is a subroutine of phase estimation that turns the periodic phase pattern on the counting register into peaks near $y = k2^t/r$; it does not factor $N$ by itself

**Concepts required.** The QFT is a component of phase estimation; Shor's algorithm is phase estimation of $U_a$ plus classical processing.

**Relevant formulae.** $\mathrm{QFT}^{-1} : \frac{1}{\sqrt{2^t}} \sum_x e^{2\pi i x\varphi} |x\rangle \mapsto\approx |2^t\varphi\rangle$.

**Step-by-step explanation.** Shor's algorithm = classical reduction + order finding (modular exponentiation, QFT, measurement) + classical post-processing. The QFT is one component; the dominant cost of real implementations is the modular exponentiation, not the QFT ($O(n^2)$ gates, or fewer when approximated).

**Why the other options are incorrect.**

**(B)** The QFT alone factors nothing; modular exponentiation and the classical steps are essential.

**(C)** $a^x$ mod $N$ is computed by the controlled modular multiplications, not by the QFT.

**(D)** No circuit acts on $|N\rangle$; $N$ enters only through the classical choice of the multipliers.

**Key takeaway.** Never confuse Shor with the QFT: the QFT is one subroutine.

**Q.54** SINGLE-CORRECT MCQ · Skipped identity multipliers

`build_shor_kernel` computes $c_j = a^{2^j}$ mod $N$ classically and skips $c_j = 1$. For $N = 15$, $a = 7$, $t = 8$, the number of controlled multiplications actually applied is

**(A)** 4

**(B)** ✓ 2 ($c_0 = 7$, $c_1 = 4$; $c_j = 1$ for $j \geq 2$)

**(C)** 1

**(D)** 8

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** 2 ($c_0 = 7$, $c_1 = 4$; $c_j = 1$ for $j \geq 2$)

**Concepts required.** The multiplier constants $c_j = a^{2^j}$ mod $N$ are computed classically; $c_j = 1$ is the identity.

**Relevant formulae.** $7, 49 \equiv 4, 16 \equiv 1, 1, \ldots$: two non-identity multipliers.

**Step-by-step explanation.** $7^1 = 7$, $7^2 = 49 \equiv 4$, $7^4 \equiv 16 \equiv 1$, and then 1 forever. Only two multipliers remain — matching the recorded gate counts (`ccustom_unitary_constant`: 2; operations `mulmod_7_mod_15`, `mulmod_4_mod_15`). This shortcut uses the known order and is specific to the tiny example.

**Why the other options are incorrect.**

**(A)** 4 is the order $r$, not the number of multipliers.

**(C)** 1 forgets that $c_1 = 4 \neq 1$ (it appears as `mulmod_4_mod_15`).

**(D)** 8 would be the count without skipping the identity multipliers; the notebook skips $c_j = 1$.

**Key takeaway.** Skipping identity multipliers exploits $r \mid 2^j$ — a shortcut available only for toy examples.

**Q.55** MATCHING LISTS · Functions of `Shor.ipynb`

Match each function of `Shor.ipynb` (List-I) with what it does (List-II).

| List-I | | List-II | |
|---|---|---|---|
| **(P)** | `mulmod_matrix` | **(1)** | brute-force order of $a$ modulo $m$ as a classical reference (feasible only for tiny $m$) |
| **(Q)** | `build_shor_kernel` | **(2)** | continued-fraction denominator of $y/2^t$ and the smallest multiple $kr'$ with $a^{kr'} \equiv 1$ |
| **(R)** | `candidate_orders` | **(3)** | builds the CUDA-Q circuit: Hadamards, controlled $U_{a^{2^j}}$, inverse QFT, measurement |
| **(S)** | `classical_order` | **(4)** | permutation matrix $\|y\rangle \mapsto \|cy \bmod N\rangle$ for $y < N$, identity above |

**(A)** P→4, Q→2, R→3, S→1 **(B)** P→4, Q→3, R→1, S→2

**(C)** ✓ P→4, Q→3, R→2, S→1 **(D)** P→3, Q→4, R→2, S→1

**Answer: (C) Detailed solution**

**Correct answer.** **(C)** P→4, Q→3, R→2, S→1

**Concepts required.** The notebook separates matrix construction, circuit construction, post-processing and classical checks.

**Relevant formulae.** `mulmod_matrix` → `cudaq.register_operation` → `build_shor_kernel` → `cudaq.sample` → `candidate_orders`.

**Step-by-step explanation.** `mulmod_matrix` builds the unitary (4); `build_shor_kernel` assembles the circuit with `cudaq.register_operation` gates (3); `candidate_orders` is the classical post-processing (2); `classical_order` is the brute-force check (1). Hence P→4, Q→3, R→2, S→1.

**Why the other options are incorrect.**

**(A)** This swaps the kernel builder and the post-processing: `build_shor_kernel` builds the circuit and never sees $y$.

**(B)** This swaps post-processing and the classical reference: `candidate_orders` uses $y$ and continued fractions; `classical_order` uses brute force.

**(D)** This swaps the matrix and the kernel builder: `mulmod_matrix` returns a NumPy permutation matrix, not a circuit.

**Key takeaway.** Know which part of the code is quantum (the kernel) and which is classical (everything around it).

**Q.56** MATCHING LISTS · Outcomes to fractions

For $N = 33$, $a = 5$ ($r = 10$) and $t = 12$, match each outcome $y$ (List-I) with `Fraction(y, 4096).limit_denominator(33)` (List-II).

| List-I | | List-II | |
|---|---|---|---|
| **(P)** | 410 | **(1)** | 2/5 |
| **(Q)** | 819 | **(2)** | 7/10 |
| **(R)** | 1638 | **(3)** | 1/10 |
| **(S)** | 2867 | **(4)** | 1/5 |

**(A)** P→4, Q→3, R→1, S→2 **(B)** P→3, Q→4, R→2, S→1

**(C)** ✓ P→3, Q→4, R→1, S→2 **(D)** P→3, Q→1, R→4, S→2

**Answer: (C) Detailed solution**

**Correct answer.** (C) P→3, Q→4, R→1, S→2
**Concepts required.** `limit_denominator(N)` returns the closest fraction with denominator at most $N$.
**Relevant formulae.** 410/4096 = 0.1001, 819/4096 = 0.19995, 1638/4096 = 0.39990, 2867/4096 = 0.69995.
**Step-by-step explanation.** 410/4096 = 0.1001 ≈ 1/10; 819/4096 = 0.19995 ≈ 1/5; 1638/4096 = 0.3999 ≈ 2/5; 2867/4096 = 0.69995 ≈ 7/10. Only 410 and 2867 give the denominator 10 directly ($s = 1, 7$ coprime to 10); 819 and 1638 give 5, and the multiples test then finds 10.
**Why the other options are incorrect.**

**(A)** This exchanges 1/10 and 1/5: 410/4096 ≈ 0.1 and 819/4096 ≈ 0.2.

**(B)** This exchanges 2/5 and 7/10: 1638/4096 ≈ 0.4 and 2867/4096 ≈ 0.7.

**(D)** This assigns 1/10 to 1638; but 1638/4096 ≈ 0.4 = 2/5.

**Key takeaway.** Only fractions with $\gcd(s, r) = 1$ give $r$ directly; others need the multiples test.

**Q.57** Assertion–Reason · Prime powers

**Assertion (A):** Shor's order-finding reduction is the method of choice for factoring $N = 81 = 3^4$.
**Reason (R):** For an odd prime power $N = p^k$, the only square roots of 1 modulo $N$ are $\pm 1$.

**(A)** Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A

**(B)** Both A and R are true, but R is *not* the correct explanation of A

**(C)** A is true, but R is false

**(D)** ✓ A is false, but R is true

**Answer: (D) Detailed solution**

**Correct answer.** (D) A is false, but R is true
**Concepts required.** Prime powers are excluded from the quantum reduction and handled classically.
**Relevant formulae.** $\mathbb{Z}_{p^k}^*$ cyclic $\Rightarrow x^2 \equiv 1 \iff x \equiv \pm 1$; $81 = 3^4$ found by integer roots.
**Step-by-step explanation.** **A is false**: prime powers are detected classically (integer $k$-th roots) — `Shor.ipynb` does so with `prime_power` before any quantum step. **R is true**: $\mathbb{Z}_{p^k}^*$ is cyclic, so $x^2 \equiv 1$ has only the solutions $\pm 1$; hence $a^{r/2}$ would always be $\pm 1$ and $\gcd(a^{r/2} \pm 1, N)$ would be trivial. Answer: (D).
**Why the other options are incorrect.**

**(A)** is wrong because A is false: prime powers are detected classically (`prime_power` in the notebook).

**(B)** is wrong because A is false, so it cannot be a true assertion with an unrelated reason.

**(C)** is wrong because A is false and R is true, the opposite of (C).

**Key takeaway.** Shor needs at least two distinct prime factors; check prime powers classically first.

**Q.58** Statement I / II · Uninformative outcomes

**Statement I:** A measured value $y = 0$ means that the order is $r = 1$.

**Statement II:** When $r$ does not divide $2^t$, every measured $y$ yields the exact order through continued fractions.

**(A)** Both Statement I and Statement II are true

**(B)** ✓ Both Statement I and Statement II are false

**(C)** Statement I is true, but Statement II is false

**(D)** Statement I is false, but Statement II is true

**Answer: (B) Detailed solution**

**Correct answer.** (B) Both Statement I and Statement II are false
**Concepts required.** Measurement outcomes are random; only some of them reveal the order directly.
**Relevant formulae.** $y = 0 \leftrightarrow s = 0$ (always present); success requires $y \approx s2^t/r$ with $\gcd(s, r) = 1$.
**Step-by-step explanation.** **I is false**: $y = 0$ is the peak $s = 0$, present for every $r$; it carries no information (the notebook returns no candidate). **II is false**: only outcomes close to $s2^t/r$ with $\gcd(s, r) = 1$ give $r$ directly; others give a divisor of $r$ or nothing, and the probability mass is spread over neighbouring $y$. Answer: (B).
**Why the other options are incorrect.**
**(A)** is wrong because both statements are false.
**(C)** is wrong because Statement I is false: $y = 0$ occurs for every order and gives no denominator.
**(D)** is wrong because Statement II is false: for $N = 21$, $a = 2$ the outcome $y = 512$ gives $1/2$, and outcomes between peaks give nothing useful.
**Key takeaway.** Shor's algorithm is probabilistic; failures are expected and handled by repetition.

**Paragraph for Questions 59 and 60**

Shor's algorithm is applied to $N = 39$ with base $a = 2$, using the register sizes of `Shor.ipynb`: the work register has $n =$ `N.bit_length()` $= 6$ qubits and the counting register $t = 2n = 12$ qubits, so the peaks of the outcome distribution lie near $y \approx k \cdot 4096/r$. A measured value $y$ is post-processed with `Fraction(y, 2**t).limit_denominator(N)`; the denominator $r'$ of the result, or the smallest multiple $kr'$ with $a^{kr'} \equiv 1 \pmod{N}$, is taken as the order. The powers of 2 modulo 39 are $2, 4, 8, 16, 32, 25, 11, 22, 5, 10, 20, 1$.

**Q.59** Paragraph based · $N = 39$: order and factors

The order $r$ and the factors $\gcd(2^{r/2} - 1, 39)$, $\gcd(2^{r/2} + 1, 39)$ are

**(A)** $r = 12$; 1 and 39
**(B)** $r = 12$; 7 and 13
**(C)** ✓ $r = 12$; 3 and 13
**(D)** $r = 6$; 3 and 13

**Answer: (C) Detailed solution**

**Correct answer.** (C) $r = 12$; 3 and 13
**Concepts required.** The order is the position of the first 1 in the list of powers.
**Relevant formulae.** $2^{12} \equiv 1 \pmod{39}$, $2^6 \equiv 25$; $\gcd(24, 39) = 3$, $\gcd(26, 39) = 13$.
**Step-by-step explanation.** From the list, $2^{12} \equiv 1$ is the first 1: $r = 12$, even. $2^6 \equiv 25 \not\equiv 38$. $\gcd(24, 39) = 3$ and $\gcd(26, 39) = 13$; $3 \times 13 = 39$. $r = 6$ would require $2^6 \equiv 1$, but $2^6 \equiv 25$.
**Why the other options are incorrect.**
**(A)** $25 \not\equiv -1 \equiv 38$, so the gcds are non-trivial.

**(B)** 7 does not divide $39 = 3 \cdot 13$.

**(D)** $2^6 \equiv 25 \neq 1$, so $r \neq 6$ (and the factors would have to come from $2^3$).

**Key takeaway.** $N = 39$, $a = 2$: $r = 12$, $39 = 3 \times 13$.

**Q.60** PARAGRAPH BASED · $N = 39$: continued fraction

The outcome $y = 1707$ is measured. The continued-fraction step gives

**(A)** ✓ $5/12$, so $r = 12$ directly

**(B)** $1/2$, so the multiples test is needed

**(C)** $5/12$, so $r = 5$

**(D)** $2/5$, so $r = 5$

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** $5/12$, so $r = 12$ directly

**Concepts required.** A continued-fraction convergent within $\frac{1}{2r^2}$ of $y/2^t$ is the reduced form of $s/r$.

**Relevant formulae.** $|1707/4096 - 5/12| = 8.1 \times 10^{-5} < 1/(2 \cdot 39^2) = 3.3 \times 10^{-4}$.

**Step-by-step explanation.** $1707/4096 = 0.41675$, and $5/12 = 0.41667$; $|1707/4096 - 5/12| = 8.1 \times 10^{-5} < 1/(2 \cdot 39^2)$, so `limit_denominator(39)` returns $5/12$. $\gcd(5, 12) = 1$, so the denominator is the order 12. The numerator 5 is the random index $s$, not the order.

**Why the other options are incorrect.**

**(B)** $1/2 = 0.5$ is far from 0.41675; that fraction belongs to $y = 2048$.

**(C)** The numerator 5 is the random index $s$; the order is the denominator 12.

**(D)** $2/5 = 0.4$ differs by 0.017, while $5/12$ differs by only $8 \times 10^{-5}$.

**Key takeaway.** Denominator = order when $\gcd(s, r) = 1$; always verify $a^r \equiv 1$.

## SECTION D · Advanced Shor's Algorithm and Implementation Q.61–80

**Q.61** Single-correct MCQ · Beauregard qubit count

Beauregard's circuit for Shor's algorithm uses $2n + 3$ logical qubits. For RSA-2048 ($n = 2048$) this is

**(A)** 2051 **(B)** 4096 **(C)** 6147 **(D)** ✓ 4099

**Answer: (D) Detailed solution**

**Correct answer.** (D) 4099
**Concepts required.** Logical-qubit counts of Shor circuits are linear in the bit length $n$.
**Relevant formulae.** Beauregard: $2n + 3$; $n = 2048$: 4099.
**Step-by-step explanation.** $2 \times 2048 + 3 = 4099$. $6147 = 3 \times 2048 + 3$ resembles the $3n$ register count of `Shor.ipynb` (6144). These are *logical* (error-corrected) qubits; the physical count is orders of magnitude larger.
**Why the other options are incorrect.**
**(A)** $2051 = n + 3$ omits one of the two $n$-qubit registers (work register and adder register).
**(B)** $4096 = 2n$ forgets the three extra qubits (carry/ancilla and the recycled control).
**(C)** $6147 = 3n + 3$; the $3n$ register count belongs to the notebook's construction, not Beauregard's.
**Key takeaway.** Thousands of logical qubits — each made of hundreds to thousands of physical qubits.

**Q.62** Single-correct MCQ · Published RSA-2048 estimates

Which pair of published resource estimates for factoring RSA-2048 is correct?

**(A)** both estimates state that RSA-2048 was factored on existing hardware
**(B)** Gidney (2025): about 20 billion qubits in about 8 hours
**(C)** ✓ Gidney and Ekerå (2019): about 20 million noisy qubits in about 8 hours; Gidney (2025): fewer than one million noisy qubits in less than a week
**(D)** Gidney and Ekerå (2019): 4099 physical qubits in one second

**Answer: (C) Detailed solution**

**Correct answer.** (C) Gidney and Ekerå (2019): about 20 million noisy qubits in about 8 hours; Gidney (2025): fewer than one million noisy qubits in less than a week
**Concepts required.** Published resource estimates are model-based projections, not experimental results.
**Relevant formulae.** GE2019: $\approx 2 \times 10^7$ physical qubits, $\approx 8$ h; Gidney 2025: $< 10^6$ physical qubits, $< 1$ week.
**Step-by-step explanation.** Both are *estimates* under explicit assumptions (surface code, physical error rate $10^{-3}$, $\mu$s cycle times), not experiments. Gidney–Ekerå 2019: $\approx 20$ million physical qubits, $\approx 8$ hours. Gidney 2025 (approximate residue arithmetic, yoked surface codes, magic-state cultivation): $< 1$ million qubits, $< 1$ week. No cryptographically relevant number has been factored by Shor's algorithm.
**Why the other options are incorrect.**
**(A)** No resource estimate claims an actual factorisation of RSA-2048; none has been performed.
**(B)** The 2025 estimate *lowered* the qubit count to under one million; 20 billion is fabricated.
**(D)** 4099 is Beauregard's *logical* count; GE2019 required about 20 million physical qubits and hours, not seconds.

**Key takeaway.** Distinguish estimates from demonstrations: RSA-2048 remains unfactored.

**Q.63** Single-correct MCQ · $O(n^3)$ scaling

With schoolbook arithmetic, modular exponentiation costs $O(n^3)$ gates. Doubling the bit length $n$ of $N$ multiplies the gate count by about

**(A)** 4 **(B)** $2^n$ **(C)** 2 **(D)** ✓ 8

**Answer: (D) Detailed solution**

**Correct answer.** **(D)** 8
**Concepts required.** Polynomial scaling: $O(n^3)$ means doubling $n$ multiplies the cost by $2^3$.
**Relevant formulae.** $(2n)^3/n^3 = 8$.
**Step-by-step explanation.** $(2n)^3 = 8n^3$. The cost is polynomial: $t = 2n$ controlled multiplications, each $O(n^2)$ gates. Classical sub-exponential algorithms (GNFS) grow much faster, which is the source of Shor's advantage — but only on a fault-tolerant machine. $2^n$ is the exponential cost of trial division.
**Why the other options are incorrect.**
**(A)** 4 would be quadratic scaling, $O(n^2)$ — e.g. the QFT alone.
**(B)** $2^n$ is exponential growth, the cost of brute-force methods, not of Shor's algorithm.
**(C)** 2 would be linear scaling, $O(n)$.
**Key takeaway.** Shor is polynomial ($O(n^3)$ or better), GNFS sub-exponential — the gap grows with $n$.

**Q.64** Single-correct MCQ · Semiclassical QFT

In the "semiclassical" (one-control-qubit) implementation of the inverse QFT used by Beauregard, the $t = 2n$ counting qubits are replaced by

**(A)** no qubits: the QFT is computed classically before the circuit
**(B)** $2n$ ancilla qubits that are never measured
**(C)** ✓ a single qubit that is measured and reset $2n$ times; earlier measurement results classically control the phase corrections
**(D)** $n$ qubits, because the QFT needs only half the bits

**Answer: (C) Detailed solution**

**Correct answer.** **(C)** a single qubit that is measured and reset $2n$ times; earlier measurement results classically control the phase corrections
**Concepts required.** The inverse QFT followed by measurement can be performed one qubit at a time with classical feed-forward.
**Relevant formulae.** Bit $y_k$ measured $\to$ rotations $R_1(-\pi \sum_j y_j 2^{-(j-k)})$ on the next qubit; $3n \to 2n+3$ qubits.
**Step-by-step explanation.** The inverse QFT followed by measurement can be done bit by bit (Griffiths–Niu): measure, then apply classically controlled phases to the next qubit. One control qubit is recycled, so the count drops from $3n$ to about $2n+3$ ($n$ work, $n+2$ ancilla for the Fourier-space adder, 1 control). The measurement statistics are unchanged.
**Why the other options are incorrect.**
**(A)** The QFT acts on quantum amplitudes, which cannot be computed classically in advance.
**(B)** Ancilla qubits that are never measured would not reduce the qubit count; the point is to recycle one qubit.
**(D)** The QFT needs all $2n$ bits of precision; they are measured sequentially on one qubit, not

reduced to $n$ qubits.

**Key takeaway.** Measurement plus classical control trades qubits for mid-circuit measurements.

**Q.65** SINGLE-CORRECT MCQ · Approximate QFT

For $N = 21$, $a = 2$, $t = 10$, an approximate inverse QFT keeps only controlled phases between qubits at distance $k - j \le m$. With $m = 2$ the number of controlled phases and the (re-executed, exact) success probability are

**(A)** 45 phases; 0.817

**(B)** 2 phases; 0.689

**(C)** 17 phases; 0.817 — truncation never changes the result

**(D)** ✓ 17 phases; 0.689 (full QFT: 45 phases, 0.817)

**Answer: (D) Detailed solution**

**Correct answer.** (D) 17 phases; 0.689 (full QFT: 45 phases, 0.817)

**Concepts required.** The approximate QFT drops small-angle controlled phases; it reduces the gate count at a small cost in accuracy.

**Relevant formulae.** Pairs at distance $\le 2$: $9 + 8 = 17$; full $\binom{10}{2} = 45$.

**Step-by-step explanation.** Distance 1: 9 pairs, distance 2: 8 pairs, total 17 (full QFT: $\binom{10}{2} = 45$). The re-executed simulation gave success 0.689 for $m = 2$, 0.787 for $m = 3$, 0.811 for $m = 4$ and 0.817 for the full QFT. Very small cut-offs lose accuracy; $m \approx \log_2 n$ plus a few suffices asymptotically.

**Why the other options are incorrect.**

**(A)** 45 phases and 0.817 describe the full QFT, not the truncation $m = 2$.

**(B)** $m = 2$ is the maximum distance, not the number of gates; each distance contributes $t - d$ pairs.

**(C)** Truncation changes the amplitudes; the re-executed simulation shows a drop from 0.817 to 0.689.

**Key takeaway.** Moderate cut-offs ($m \approx 4$) lose little; aggressive ones lose a lot.

**Q.66** SINGLE-CORRECT MCQ · Controlled phases in the QFT

The full inverse QFT on $t = 8$ counting qubits (notebook default for $N = 15$) contains how many controlled phase gates?

**(A)** 8 **(B)** 64 **(C)** ✓ 28 **(D)** 56

**Answer: (C) Detailed solution**

**Correct answer.** (C) 28

**Concepts required.** The QFT on $t$ qubits uses one controlled phase per qubit pair.

**Relevant formulae.** $t(t-1)/2 = 28$ for $t = 8$.

**Step-by-step explanation.** One for each pair $j < k$: $\binom{8}{2} = \frac{8 \cdot 7}{2} = 28$ — the recorded gate count `cr1: 28`. In general $t(t-1)/2 = O(t^2)$, which for large $t$ is dwarfed by the $O(n^3)$ cost of the modular exponentiation.

**Why the other options are incorrect.**

**(A)** 8 is the number of Hadamards in the inverse QFT, not of controlled phases.

**(B)** $64 = t^2$ counts ordered pairs including $j = k$.

**(D)** $56 = t(t-1)$ counts each unordered pair twice.

**Key takeaway.** QFT cost is $O(t^2)$ — small compared with modular exponentiation.

**Q.67** SINGLE-CORRECT MCQ · Recorded gate counts

The recorded gate counts of `Shor.ipynb` for $N = 15$ are `x: 1, ccustom_unitary_constant: 2, swap: 4, h: 16, cr1: 28` (and `mz: 8`). Why are there 16 Hadamard gates?

**(A)** each controlled multiplication contains 8 Hadamards

**(B)** Hadamards are inserted before every measurement of the work register

**(C)** $16 = 2^4$ is the dimension of the work register

**(D)** ✓ 8 create the superposition on the counting qubits and 8 belong to the inverse QFT

**Answer: (D) Detailed solution**

**Correct answer.** **(D)** 8 create the superposition on the counting qubits and 8 belong to the inverse QFT

**Concepts required.** Hadamards appear at the start (superposition) and in the inverse QFT (one per qubit).

**Relevant formulae.** $8 + 8 = 16$; total $1 + 2 + 4 + 16 + 28 = 51$ gates without `mz`.

**Step-by-step explanation.** $t = 8$ Hadamards start the algorithm, and the inverse QFT applies one $H$ per qubit (8 more). The multipliers are single custom gates (no internal $H$), and the work register is not measured. Total gates without `mz`: $1 + 2 + 4 + 16 + 28 = 51$, depth 20.

**Why the other options are incorrect.**

**(A)** The multipliers are single `ccustom_unitary_constant` gates; no Hadamards are inside them.

**(B)** The work register is not measured in the notebook; only `mz(count)` appears.

**(C)** The coincidence $16 = 2^4$ is irrelevant; the count is $t + t$.

**Key takeaway.** Gate counts should be traced back to the circuit's structure.

**Q.68** SINGLE-CORRECT MCQ · No hardware execution

Why does `Shor.ipynb` refuse to run on hardware targets?

**(A)** the inverse QFT cannot be executed on hardware

**(B)** Shor's algorithm is mathematically impossible on hardware

**(C)** CUDA-Q has no hardware targets

**(D)** ✓ its modular multipliers are exact custom unitaries (permutation matrices) registered with `cudaq.register_operation`; hardware would need them decomposed into native gates, which the notebook does not do

**Answer: (D) Detailed solution**

**Correct answer.** **(D)** its modular multipliers are exact custom unitaries (permutation matrices) registered with `cudaq.register_operation`; hardware would need them decomposed into native gates, which the notebook does not do

**Concepts required.** Custom matrix gates are a simulator convenience; hardware executes only native gate sets.

**Relevant formulae.** `cudaq.register_operation(name, matrix)`; hardware → `ValueError` in the notebook.

**Step-by-step explanation.** The notebook raises a `ValueError` for hardware and points to the resource estimates. A dense $2^n \times 2^n$ matrix gate is fine for a simulator but has no native implementation; compiling it costs $O(n^2)$–$O(n^3)$ gates per multiplier. The inverse QFT itself uses standard gates.

**Why the other options are incorrect.**

**(A)** The inverse QFT uses $H$, controlled phases and SWAPs, which compile to native gates.

**(B)** Shor's algorithm is a well-defined circuit; small instances have run on hardware with compiled (often simplified) circuits.

**(C)** CUDA-Q supports several hardware providers; the restriction is the notebook's custom gates.
**Key takeaway.** Running on hardware requires a gate-level multiplier — the hard engineering part.

**Q.69** SINGLE-CORRECT MCQ · CNOTs of a controlled multiplier

In the gate-level circuit, "multiply by 7 modulo 15" is built as three left rotations of the 4 work bits (each 3 SWAPs) followed by $X$ on all four bits. Its controlled version, with controlled SWAPs of 8 CNOTs and controlled-$X$ = CNOT, uses how many CNOTs?

**(A)** 9 **(B)** 40 **(C)** ✓ 76 **(D)** 72

**Answer: (C) Detailed solution**

**Correct answer.** (C) 76
**Concepts required.** Count CNOTs gate by gate: controlled SWAP = 8, controlled $X$ = 1.
**Relevant formulae.** $9 \times 8 + 4 = 76$; whole circuit: $76 + 48 + 3 = 127$.
**Step-by-step explanation.** 9 controlled SWAPs $\times$ 8 = 72 CNOTs, plus 4 CNOTs for the controlled $X$ gates: 76. (Multiplication by 7 is valid on $y = 1, \ldots, 14$: $7y \bmod 15 = 15 - (y \cdot 2^3 \bmod 15)$.) With the controlled $\times 4$ (6 controlled SWAPs = 48) and the QFT SWAP (3) the circuit has 127 CNOTs.
**Why the other options are incorrect.**
**(A)** 9 counts controlled SWAPs, not CNOTs.
**(B)** 40 would use 4 CNOTs per controlled SWAP; a Toffoli alone needs 6.
**(D)** 72 forgets the four controlled $X$ gates.
**Key takeaway.** Even multiplying a 4-bit number by 7 costs 76 CNOTs when controlled.

**Q.70** SINGLE-CORRECT MCQ · Random baseline

In the gate-level circuit ($t = 3$, $a = 7$, $N = 15$) the counting outcome becomes uniformly random in the fully depolarized limit. The success probability with the notebook's post-processing is then

**(A)** 0
**(B)** ✓ 3/8: $y = 2, 4, 6$ give $r = 4$ and the factors; $y = 0$ gives nothing; odd $y$ give $r' = 8$, and $7^4 \equiv 1$ gives only trivial factors
**(C)** 3/4, the same as without noise
**(D)** 1/8

**Answer: (B) Detailed solution**

**Correct answer.** (B) 3/8: $y = 2, 4, 6$ give $r = 4$ and the factors; $y = 0$ gives nothing; odd $y$ give $r' = 8$, and $7^4 \equiv 1$ gives only trivial factors
**Concepts required.** A uniformly random outcome is the zero-information baseline for any noisy run.
**Relevant formulae.** Successful $y \in \{2, 4, 6\}$: $3/8 = 0.375$; odd $y \to 8$, $7^4 \equiv 1 \to$ trivial gcds.
**Step-by-step explanation.** $y/8$: $2/8 = 1/4$, $6/8 = 3/4$ give 4; $4/8 = 1/2$ gives 4 via multiples. For odd $y$ the denominator 8 satisfies $7^8 \equiv 1$, but $7^4 \equiv 1$, so $\gcd(0, 15) = 15$, $\gcd(2, 15) = 1$ — trivial. Hence $3/8 = 0.375$: a noisy run can still "succeed" by luck, which is why a success rate must be compared with this random baseline.
**Why the other options are incorrect.**
**(A)** 0 ignores that random outcomes can still give $r = 4$ by chance.
**(C)** 3/4 is the noiseless success probability; full depolarization destroys the peaks.
**(D)** 1/8 counts only one of the three successful outcomes.

**Key takeaway.** Compare noisy success rates with the random baseline, never with zero.

**Q.71** Single-correct MCQ · Memory for $N = 1023$

Using the notebook's $3n$-qubit construction, the state-vector memory (16 bytes per amplitude) for $N = 1023$ is about

**(A)** 268 MB (24 qubits) **(B)** ✓ 17.2 GB (30 qubits)

**(C)** 1.1 TB (36 qubits) **(D)** 16.4 kB

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** 17.2 GB (30 qubits)
**Concepts required.** Memory of the full state vector: $16 \cdot 2^{3n}$ bytes in the notebook's construction.
**Relevant formulae.** 1023: $n = 10$, $q = 30$, $M = 16 \cdot 2^{30} = 17.2$ GB.
**Step-by-step explanation.** 1023 has 10 bits, so $3n = 30$ qubits: $16 \times 2^{30} = 17.18 \times 10^9$ bytes. This matches the notebook's reverse estimate that 80 GB allows at most 32 qubits, i.e. $N \leq 1023$. $N = 1024$ would need 11 bits and 33 qubits (137 GB).
**Why the other options are incorrect.**

**(A)** 268 MB corresponds to 24 qubits, i.e. an 8-bit $N$ such as 143.

**(C)** 1.1 TB corresponds to 36 qubits, i.e. a 12-bit $N$.

**(D)** 16.4 kB is $16 \cdot 2^{10}$: it uses $n$ qubits instead of $3n$.

**Key takeaway.** Simulation memory explodes: a few more bits of $N$ exhaust any computer.

**Q.72** Single-correct MCQ · Toy RSA

Toy RSA: $p = 5$, $q = 11$, $e = 7$. The private exponent $d$ and the ciphertext of $m = 2$ are

**(A)** ✓ $d = 23$; $c = 18$ **(B)** $d = 23$; $c = 14$ **(C)** $d = 3$; $c = 18$ **(D)** $d = 7$; $c = 2$

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** $d = 23$; $c = 18$
**Concepts required.** RSA: $ed \equiv 1 \pmod{\varphi(N)}$, $c = m^e \bmod N$, $m = c^d \bmod N$.
**Relevant formulae.** $\varphi(55) = 40$, $7 \cdot 23 = 161 \equiv 1$; $2^7 = 128 \equiv 18 \pmod{55}$.
**Step-by-step explanation.** $\varphi(55) = 4 \cdot 10 = 40$; $7d \equiv 1 \pmod{40}$: $7 \cdot 23 = 161 = 4 \cdot 40 + 1$, so $d = 23$. $c = 2^7 \bmod 55 = 128 - 110 = 18$; check $18^{23} \bmod 55 = 2$. Shor's algorithm breaks RSA by factoring $N = 55$, which reveals $\varphi(N)$ and hence $d$.
**Why the other options are incorrect.**

**(B)** $c = 2^7 \bmod 55 = 18$, not 14.

**(C)** $7 \cdot 3 = 21 \not\equiv 1 \pmod{40}$, so $d \neq 3$.

**(D)** $d = e$ would need $49 \equiv 1 \pmod{40}$; and encryption does change the message.

**Key takeaway.** Factoring $N$ gives $\varphi(N)$ and hence $d$: that is the threat.

**Q.73** Single-correct MCQ · Post-quantum standards

Which standard specifies NIST's lattice-based key-encapsulation mechanism intended to replace RSA/ECDH key exchange?

**(A)** FIPS 197 (AES)
**(B)** ✓ FIPS 203 (ML-KEM)
**(C)** SP 800-90A (random bit generators)
**(D)** FIPS 186 (DSA/ECDSA)

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** FIPS 203 (ML-KEM)
**Concepts required.** NIST's post-quantum standards: FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA), 2024.
**Relevant formulae.** ML-KEM: module learning with errors; no known efficient quantum algorithm.
**Step-by-step explanation.** NIST published FIPS 203 (ML-KEM, from CRYSTALS-Kyber), FIPS 204 (ML-DSA, from Dilithium) and FIPS 205 (SLH-DSA, from SPHINCS+) in August 2024. AES (FIPS 197) is symmetric and not broken by Shor; FIPS 186 contains the signature schemes Shor's algorithm does break.
**Why the other options are incorrect.**
**(A)** FIPS 197 is AES, a symmetric cipher; it is not a key-encapsulation mechanism.
**(C)** SP 800-90A specifies random bit generators, not key exchange.
**(D)** FIPS 186 specifies DSA/ECDSA/RSA signatures — schemes broken by Shor's algorithm.
**Key takeaway.** Replace RSA/ECC key exchange by ML-KEM (often in hybrid mode).

**Q.74** Single-correct MCQ · Impact on RSA, ECC, AES

Which statement about the cryptographic impact of a large fault-tolerant quantum computer is correct?

**(A)** ✓ Shor's algorithm breaks RSA and elliptic-curve cryptography; AES is affected only by Grover's quadratic speed-up, so AES-256 remains adequate
**(B)** hash functions such as SHA-256 are broken by Shor's algorithm
**(C)** Shor's algorithm breaks AES-256 in polynomial time
**(D)** only RSA is affected; elliptic-curve cryptography is immune

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** Shor's algorithm breaks RSA and elliptic-curve cryptography; AES is affected only by Grover's quadratic speed-up, so AES-256 remains adequate
**Concepts required.** Shor solves the hidden subgroup problem in abelian groups: factoring and discrete logarithms, including on elliptic curves.
**Relevant formulae.** Grover: $O(\sqrt{2^k})$ for a $k$-bit key: AES-256 $\to\approx 2^{128}$ work.
**Step-by-step explanation.** Shor's period finding solves factoring and discrete logarithms in any abelian group, including elliptic-curve groups (ECDH, ECDSA). Symmetric ciphers and hash functions have no such structure; Grover search gives at most a square-root speed-up, mitigated by doubling key length.
**Why the other options are incorrect.**
**(B)** Hash functions are affected only by generic (Grover-type) speed-ups, not by Shor.
**(C)** Shor's algorithm needs algebraic periodicity; AES has none, and Grover gives only a square-root speed-up.
**(D)** ECC relies on discrete logarithms, which Shor's algorithm also solves (with fewer qubits than RSA of equal security).

**Key takeaway.** Public-key cryptography falls; symmetric cryptography needs at most larger keys.

**Q.75** MATCHING LISTS · Fault-tolerance terms

Match each fault-tolerance term (List-I) with its meaning (List-II).

| List-I | | List-II | |
|---|---|---|---|
| **(P)** | logical qubit | **(1)** | minimum number of physical errors that can cause an undetected logical error |
| **(Q)** | physical qubit | **(2)** | a special resource state that enables non-Clifford gates such as $T$ or Toffoli in the surface code |
| **(R)** | code distance $d$ | **(3)** | an error-corrected qubit encoded in many noisy qubits |
| **(S)** | magic state | **(4)** | a noisy hardware qubit with error rates around $10^{-3}$ |

**(A)** P→3, Q→4, R→2, S→1
**(B)** ✓ P→3, Q→4, R→1, S→2
**(C)** P→4, Q→3, R→1, S→2
**(D)** P→3, Q→1, R→4, S→2

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** P→3, Q→4, R→1, S→2
**Concepts required.** Fault tolerance encodes logical qubits in many physical qubits and supplies non-Clifford gates via magic states.
**Relevant formulae.** Surface code: $\approx 2d^2$ physical qubits per logical qubit; logical error $\sim (p/p_{\text{th}})^{(d+1)/2}$.
**Step-by-step explanation.** Logical qubit: encoded, error-corrected (3); physical qubit: the noisy hardware element (4); code distance: the number of errors needed for an undetected logical error — a surface-code patch uses about $2d^2$ physical qubits (1); magic states: distilled or cultivated resources for $T$/Toffoli (2). Shor's arithmetic is dominated by Toffoli gates, so magic-state production dominates its cost.
**Why the other options are incorrect.**

**(A)** This swaps code distance and magic states; the distance is a number of errors, magic states are resources.

**(C)** This swaps logical and physical qubits; the logical qubit is the encoded one.

**(D)** This gives the physical qubit the meaning of code distance; a physical qubit is the noisy hardware element.

**Key takeaway.** Shor's cost on real hardware is dominated by error correction and magic states.

**Q.76** MATCHING LISTS · Qubit counts of constructions

Match each construction for an $n$-bit $N$ (List-I) with its logical-qubit count (List-II).

| List-I | | List-II | |
|---|---|---|---|
| **(P)** | `Shor.ipynb` ($t = 2n$ counting + $n$ work) | **(1)** | $n + 1$ |
| **(Q)** | Beauregard (2003) | **(2)** | $3n + 0.002\, n \log_2 n$ |
| **(R)** | Gidney–Ekerå (2019) | **(3)** | $3n$ |
| **(S)** | one recycled counting qubit + $n$ work qubits with exact (oracle) multipliers | **(4)** | $2n + 3$ |

**(A)** ✓ P→3, Q→4, R→2, S→1 **(B)** P→4, Q→3, R→2, S→1

**(C)** P→3, Q→4, R→1, S→2 **(D)** P→3, Q→2, R→4, S→1

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** P→3, Q→4, R→2, S→1

**Concepts required.** Different constructions trade qubits for gates and measurements.

**Relevant formulae.** $3n$ (notebook), $2n + 3$ (Beauregard), $3n + 0.002n \log_2 n$ (GE2019), $n + 1$ (idealised semiclassical).

**Step-by-step explanation.** Notebook: $2n + n = 3n$ (3). Beauregard: $2n + 3$ (4). Gidney–Ekerå: $3n + 0.002n \log_2 n$, about 6190 for $n = 2048$ (2). With the semiclassical QFT and idealised matrix multipliers one needs only $n + 1$ (1) — but real multipliers need ancilla qubits.

**Why the other options are incorrect.**

**(B)** This swaps the notebook and Beauregard: the notebook uses $t + n = 3n$.

**(C)** This swaps Gidney–Ekerå and the idealised semiclassical count.

**(D)** This swaps Beauregard and Gidney–Ekerå; GE2019 uses slightly more than $3n$ qubits but far fewer gates.

**Key takeaway.** Qubit counts alone do not decide feasibility; gate counts and error rates matter too.

**Q.77** ASSERTION–REASON · Harvest now, decrypt later

**Assertion (A):** Migrating to post-quantum cryptography is urgent even though no current quantum computer can run Shor's algorithm at cryptographic sizes.

**Reason (R):** Shor's algorithm also solves the discrete-logarithm problem.

**(A)** Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A

**(B)** ✓ Both A and R are true, but R is *not* the correct explanation of A

**(C)** A is true, but R is false

**(D)** A is false, but R is true

**Answer: (B) Detailed solution**

**Correct answer.** **(B)** Both A and R are true, but R is *not* the correct explanation of A

**Concepts required.** Data with long confidentiality lifetimes is already at risk from future quantum computers.

**Relevant formulae.** Urgent if (data lifetime) + (migration time) > (time to a cryptographically relevant quantum computer).

**Step-by-step explanation.** **A is true**: encrypted data recorded today can be decrypted later

("harvest now, decrypt later"), and migrations take years. **R is true**: period finding solves discrete logarithms, so Diffie–Hellman and ECC fall too. But R does not explain the *urgency* — that comes from data lifetime and migration time. Answer: (B).

**Why the other options are incorrect.**

**(A)** is wrong because R, although true, does not explain the urgency; that comes from harvest-now-decrypt-later and slow migration.

**(C)** is wrong because R is true: Shor's algorithm solves discrete logarithms.

**(D)** is wrong because A is true.

**Key takeaway.** The threat is about the future decryption of today's recorded data.

**Q.78** STATEMENT I / II · Qubits and repeated squaring

**Statement I:** In the notebook's construction, a 32-bit $N$ needs 96 qubits.

**Statement II:** Repeated squaring lets $a^x \bmod N$ be implemented with $t$ controlled multiplications by the classically precomputed constants $a^{2^j} \bmod N$.

**(A)** ✓ Both Statement I and Statement II are true

**(B)** Both Statement I and Statement II are false

**(C)** Statement I is true, but Statement II is false

**(D)** Statement I is false, but Statement II is true

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** Both Statement I and Statement II are true

**Concepts required.** Qubit count $3n$ and the repeated-squaring structure of modular exponentiation.

**Relevant formulae.** $3 \cdot 32 = 96$; $a^x = \prod_{j=0}^{t-1} (a^{2^j})^{x_j}$.

**Step-by-step explanation.** **I is true**: $3n = 96$ (the notebook's reverse estimate: 100 qubits allow 33 bits). **II is true**: $a^x = \prod_j (a^{2^j})^{x_j}$, so counting qubit $j$ controls the multiplication by $a^{2^j} \bmod N$, computed classically. Answer: (A).

**Why the other options are incorrect.**

**(B)** is wrong because both statements are true.

**(C)** is wrong because Statement II is true: the controlled multipliers by $a^{2^j}$ are exactly the notebook's construction.

**(D)** is wrong because Statement I is true: $t + n = 64 + 32 = 96$.

**Key takeaway.** $t$ controlled multiplications with classically precomputed constants implement $a^x \bmod N$.

**Paragraph for Questions 79 and 80**

A gate-level version of the order-finding circuit for $N = 15$, $a = 7$ uses $t = 3$ counting qubits and 4 work qubits. The multiplications by 7 and $4 = 7^2 \bmod 15$ are compiled into controlled SWAPs (each CNOT–Toffoli–CNOT, Toffoli = 6 CNOTs) and CNOTs; the inverse QFT uses 3 CNOTs (one SWAP) and 3 controlled phases. In total the circuit has 127 CNOTs. It was simulated with the CUDA-Q `density-matrix-cpu` target and a depolarizing noise model: a one-qubit channel with parameter $p$ after every $H$, $R_z$ and $X$, and a two-qubit channel with parameter $p$ after every CNOT and controlled phase. The probability that one shot yields the factors $3 \times 5$ was (re-executed, exact from the density matrix):

| $p$ | 0 | 0.001 | 0.003 | 0.01 | 0.03 |
|---|---|---|---|---|---|
| success probability | 0.750 | 0.740 | 0.726 | 0.703 | 0.665 |

If the counting register were completely random (uniform $y$), the success probability would be 0.375. These are simulator results, not measurements on a quantum processor.

**Q.79** PARAGRAPH BASED · Error-free probability vs success

At $p = 0.01$, the probability that *all* 127 CNOTs are free of error is about $0.99^{127}$. Compare it with the success probability.

**(A)** ✓ $0.99^{127} \approx 0.28$, yet the success probability is 0.703: many errors do not change $y$ or still lead to a correct order

**(B)** $0.99^{127} \approx 0.28$, so the success probability must be at most 0.28

**(C)** $0.99^{127} \approx 0.88$, equal to the success probability

**(D)** $0.99^{127} \approx 0.01$, so the result is pure noise

**Answer: (A) Detailed solution**

**Correct answer.** **(A)** $0.99^{127} \approx 0.28$, yet the success probability is 0.703: many errors do not change $y$ or still lead to a correct order

**Concepts required.** The error-free probability bounds nothing: many errors are harmless for the final answer.

**Relevant formulae.** $0.99^{127} = e^{-1.276} = 0.279$; measured (simulated) success 0.703.

**Step-by-step explanation.** $0.99^{127} = e^{127 \ln 0.99} = e^{-1.276} = 0.279$. The success probability is not bounded by the error-free probability: errors on work qubits after their last use, phase errors that commute with measurement, or errors that shift $y$ to another successful value do not cause failure. Also, a random $y$ still succeeds with probability 0.375.

**Why the other options are incorrect.**

**(B)** Errors that do not change $y$, or move it to another successful value, do not cause failure; $0.703 > 0.28$ contradicts this option.

**(C)** $0.99^{127} \approx 0.28$, not 0.88; $0.88 \approx 0.99^{13}$.

**(D)** $0.99^{127} \approx 0.28$, not 0.01; and 0.703 is well above the random baseline 0.375.

**Key takeaway.** Estimate error impact by simulation, not by the naive product of gate fidelities.

**Q.80** PARAGRAPH BASED · Interpreting noisy results

Which conclusion is supported by the table?

**(A)** noise increases the success probability

**(B)** the results show that a quantum processor factored 15

**(C)** ✓ the success probability decreases monotonically from 0.750 towards the random baseline 0.375; at $p = 0.03$ it is 0.665, still above the baseline

**(D)** at $p = 0.03$ the circuit is completely random

**Answer: (C) Detailed solution**

**Correct answer.** (C) the success probability decreases monotonically from 0.750 towards the random baseline 0.375; at $p = 0.03$ it is 0.665, still above the baseline

**Concepts required.** Noise interpolates between the ideal distribution and the uniform baseline.

**Relevant formulae.** Ideal 0.750; baseline 0.375; $p = 0.03$: 0.665.

**Step-by-step explanation.** The values fall $0.750 \to 0.740 \to 0.726 \to 0.703 \to 0.665$; the gap to the uniform baseline (0.375) shrinks but has not vanished. These are density-matrix *simulations* with an idealised noise model — no quantum processor was used, and factoring 15 with a compiled circuit that uses the known order is a demonstration, not evidence of quantum advantage.

**Why the other options are incorrect.**

**(A)** The values decrease with $p$; noise never helped here.

**(B)** These are simulations on `density-matrix-cpu`; no quantum processor was used.

**(D)** 0.665 is far above the random baseline 0.375; the circuit is not yet completely random.

**Key takeaway.** Label simulated results as simulations; compare them with the right baseline.